

# 1 Appendix H: Spectral Indicators and Advisory Invariants

## H.1. Scope and non-Core status

This appendix records optional spectral indicators and advisory invariants that may be used to guide inspection, prioritize candidate windows, detect suspicious regions, or organize Search-side exploration.

The material here belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Advisory invariants may guide attention, but they do not determine Core verdicts.

In particular, no spectral indicator, scalar potential, score, warning flag, advisory stable band, proxy, or AI-generated confidence value may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_\tau.$$

All Core-relevant persistence data remain governed by the repaired windowed evaluation:

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

**Remark 1.1** (Technical extension status). The quantities introduced in this appendix are not Core gates. They are optional technical aids. They may be useful for detecting suspicious behavior, identifying regions worth rechecking, organizing threshold sweeps, or prioritizing Search runs, but they do not certify, discharge, repair, or override Core conditions.

**Remark 1.2** (Relation to Core sovereignty). Chapter 6 established Core sovereignty: Search-side and advisory quantities do not update Core pass/reject verdicts. This appendix obeys that doctrine. All indicators introduced here are read as proposer-side or advisory-side data unless explicitly re-entered into the Core verifier through repaired evaluation records, declared gates, monitored diagnostics, tower comparison artifacts, and budget checks.

**Remark 1.3** (Relation to Appendix G). Appendix G allows advisory attachments to certificate records only as non-verdict data. Accordingly, any spectral indicator or advisory invariant recorded in a manifest must be marked as advisory unless it has been explicitly converted into verifier-side data, such as a declared budget component or a Core diagnostic artifact.

## H.2. Advisory invariant principle

**Definition 1.4** (Advisory invariant). An *advisory invariant* is a quantity attached to a persistence object, filtered object, window, threshold, tower, comparison artifact, interface object, or run whose purpose is to guide inspection or flag potential instability, but not to determine Core acceptance.

**Definition 1.5** (Advisory reading rule). An advisory invariant  $A_{\text{adv}}$  is interpreted only as:

$$A_{\text{adv}} \rightsquigarrow \text{attention, warning, prioritization, navigation, or rechecking.}$$

It is never interpreted as:

$$A_{\text{adv}} \rightsquigarrow \text{Core proof, Core discharge, Core pass, or Core reject.}$$

**Definition 1.6** (Verifier-side re-entry). An advisory invariant *re-enters* the verifier side only if a separate Core-admissible policy converts it into one of the following:

(i) a repaired evaluation record

$$\text{Eval}_{i,w,\tau}(F);$$

(ii) an admissible representative record

$$C_{\tau,w}F;$$

(iii) an eligibility record for an invoked  $\text{Ext}^1$ -clause;

(iv) a declared tower comparison artifact

$$\phi_{i,w,\tau} : \text{ColimProfile}_{i,w,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,w,\tau}(F_\infty);$$

(v) a monitored diagnostic record;

(vi) a declared  $\delta$ -ledger component;

(vii) a gate verdict record.

**Proposition 1.7** (Advisory invariants do not update Core verdicts). *Let  $A_{\text{adv}}$  be any advisory invariant introduced in this appendix. Then  $A_{\text{adv}}$ , by itself, does not alter the Core pass/reject verdict of a declared certificate record.*

*Proof.* By Chapter 6 and Appendix G, Core verdicts are determined only by verifier-side data: repaired evaluation records, eligibility records when invoked, monitored diagnostics, declared tower comparison artifacts, gate verdicts, and the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

An advisory invariant is not one of these verifier-side conditions unless it has been explicitly converted by a declared verifier-side re-entry policy. Therefore it has no authority to update the Core verdict.  $\square$

**Remark 1.8** (Advisory does not mean useless). The fact that advisory invariants do not determine Core verdicts does not make them irrelevant. They may be highly useful for efficient exploration, anomaly detection, diagnostic triage, threshold selection, and selection of windows for further audited verification. Their role is operationally valuable but logically subordinate.

### H.3. Spectral indicators

**Definition 1.9** (Spectral indicator). A *spectral indicator* is an advisory invariant derived from spectral, graded, filtered, barcode, energy-like, or threshold-sweep data associated with a filtered object, persistence object, tower, or repaired evaluation record.

Typical examples include quantities measuring:

- (i) concentration of bars in selected length bands;
- (ii) spectral slope or decay;
- (iii) instability near a threshold;
- (iv) distributional irregularity of persistence mass;
- (v) discrepancy between adjacent window spectra;
- (vi) warning-level behavior in a threshold sweep;

(vii) sensitivity of residual bar structure near the deletion threshold  $\tau$ .

**Definition 1.10** (Spectral profile). Let  $F$  be a declared filtered object and  $W$  a window. A *spectral profile* is an advisory function

$$\text{Spec}_{F,W} : (0, \infty) \rightarrow \mathbb{R}_{\geq 0} \cup \{\infty\}$$

whose value at a scale parameter  $r$  records a declared spectral or persistence-derived quantity at scale  $r$ .

**Definition 1.11** (Repaired-profile spectral indicator). A spectral indicator is *repaired-profile based* if it is computed from the repaired windowed collapsed profile

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F))$$

or from a declared family of such profiles over thresholds, windows, or monitored degrees.

**Definition 1.12** (Band-limited spectral indicator). Given a spectral profile  $\text{Spec}_{F,W}$  and a scale interval

$$B = [r_0, r_1],$$

the associated *band-limited spectral indicator* is a declared quantity of the form

$$\text{Sl}_B(F, W) := \int_{r_0}^{r_1} \text{Spec}_{F,W}(r) dr,$$

or a discrete analogue

$$\text{Sl}_B^{\text{disc}}(F, W) := \sum_{r_j \in B} \text{Spec}_{F,W}(r_j) \Delta r_j.$$

**Remark 1.13** (No canonical spectral profile). This appendix does not impose a unique spectral profile. Different technical settings may choose different profiles. The only Core-level requirement is that such profiles remain advisory unless their use is explicitly translated into verifier-side data through the repaired Core schema.

**Remark 1.14** (Spectral indicators are not persistence profiles). A spectral indicator may be computed from persistence data, but it is not itself the repaired Core profile

$$\text{Eval}_{i,W,\tau}(F).$$

Only the latter is used directly in the topological clause of B-Gate<sup>+</sup>.

#### H.4. Stability-only interpretation

**Definition 1.15** (Stability-only interpretation). A spectral indicator has a *stability-only interpretation* if it is used only to estimate sensitivity, identify unstable regions, or prioritize rechecking, without serving as an acceptance predicate.

**Proposition 1.16** (Spectral stability does not imply Core acceptance). *Suppose a spectral indicator appears stable across a declared threshold band, window family, or monitored parameter range. This does not imply that B-Gate<sup>+</sup> passes.*

*Proof.* B-Gate<sup>+</sup> requires:

$$\text{Eval}_{1,W,\tau}(F) = 0,$$

eligible categorical vanishing when the  $\text{Ext}^1$ -clause is invoked,

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0),$$

declared tower comparison artifacts when Type IV diagnostics are invoked, and

$$\Sigma\delta < \text{gap}_\tau.$$

Spectral stability is not identical to any of these verifier-side conditions. Therefore spectral stability alone cannot imply Core acceptance.  $\square$

**Proposition 1.17** (Spectral instability does not automatically imply Core rejection). *Suppose a spectral indicator appears unstable. This does not automatically imply Core rejection.*

*Proof.* Core rejection occurs only when a Core evaluation, gate, monitored diagnostic, artifact, or budget condition fails. Spectral instability may suggest that such a failure should be checked, but it is not itself a Core failure condition.  $\square$

**Remark 1.18** (Correct use of spectral instability). Spectral instability should be used as a warning flag:

$$\text{spectral instability} \rightsquigarrow \text{run or inspect Core checks more carefully.}$$

It should not be used as:

$$\text{spectral instability} \rightsquigarrow \text{Core reject.}$$

## H.5. Relation to B-Gate<sup>+</sup>

**Definition 1.19** (Gate-adjacent advisory quantity). An advisory invariant is *gate-adjacent* if it is designed to correlate with one of the clauses of B-Gate<sup>+</sup>, such as:

(i) possible residual degree-1 persistence:

$$\text{Eval}_{1,W,\tau}(F) \neq 0;$$

(ii) possible categorical obstruction:

$$\text{Ext}^1(\mathcal{R}(C_{\tau,W}F), k) \neq 0;$$

(iii) possible monitored Type IV defect:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \neq (0, 0);$$

(iv) possible budget sensitivity:

$$\Sigma\delta \approx \text{gap}_\tau.$$

**Proposition 1.20** (Gate-adjacent is not gate-equivalent). *A gate-adjacent advisory invariant is not equivalent to the corresponding Core gate clause unless an explicit Core theorem proves such an equivalence.*

*Proof.* By definition, a gate-adjacent quantity is designed to correlate with a gate clause. Correlation or heuristic alignment is weaker than logical equivalence. Without a Core theorem establishing equivalence, the advisory quantity remains non-verdict data.  $\square$

**Remark 1.21** (No replacement of B-Gate<sup>+</sup>). No spectral indicator in this appendix replaces B-Gate<sup>+</sup>. At most, it may recommend where B-Gate<sup>+</sup> should be evaluated or re-evaluated.

**Remark 1.22** (Topological gate uses Eval, not an advisory score). The repaired topological gate is:

$$\text{Eval}_{1,w,\tau}(F) = 0.$$

It is not the vanishing of a spectral score, warning flag, potential value, or advisory indicator unless a separate theorem proves exact equivalence and the result is re-entered through the verifier-side manifest.

## H.6. Relation to $\Phi$ and navigation quantities

**Definition 1.23** (Navigation quantity). A *navigation quantity* is an advisory scalar, vector, map, or profile used to guide exploration, candidate selection, or prioritization. The scalar potential

$$\Phi$$

is treated as a navigation quantity.

**Definition 1.24** (Potential landscape). A *potential landscape* is a function

$$\Phi : \mathcal{X} \rightarrow \mathbb{R} \cup \{\infty\}$$

on a declared search or parameter space  $\mathcal{X}$ , used to rank or prioritize candidate regions for further Core verification.

**Proposition 1.25** ( $\Phi$  is not a certificate). *The value of  $\Phi$  at a candidate region does not certify that region. It only assigns advisory priority.*

*Proof.* Certification requires a Core certificate record satisfying repaired evaluation, gate, diagnostic, artifact, and budget conditions. The value of  $\Phi$  is not a repaired evaluation record, not a gate verdict, not a monitored diagnostic, not a tower comparison artifact, and not a budget comparison. Therefore it is not a certificate.  $\square$

**Remark 1.26** (Low  $\Phi$  and high  $\Phi$ ). A low value of  $\Phi$  may indicate a promising region. A high value of  $\Phi$  may indicate a difficult or suspicious region. Neither interpretation is a Core theorem. Both are advisory readings.

## H.7. Spectral warning flags

**Definition 1.27** (Spectral warning flag). A *spectral warning flag* is an advisory signal raised when a spectral indicator exceeds, falls below, oscillates, or changes rapidly across a declared advisory threshold, window, or threshold band.

**Definition 1.28** (Warning flag policy). A *warning flag policy* specifies:

- (i) the advisory indicator being monitored;
- (ii) the advisory threshold or change criterion;
- (iii) the recommended response;
- (iv) whether the response is rechecking, refinement, resampling, representative review, tower artifact review, or escalation to Core verification;

(v) the fact that the flag is not itself a Core verdict.

**Proposition 1.29** (Warning flag triggers inspection, not verdict). *A spectral warning flag may trigger additional inspection or Core verification, but it does not determine pass or reject.*

*Proof.* A warning flag is an advisory signal by Definition 1.27. Core verdicts are determined only by the verifier-side conditions of Chapter 6 and Appendix G. Therefore the warning flag can trigger inspection but not verdict.  $\square$

**Remark 1.30** (Recommended interpretation). The recommended interpretation is:

warning flag  $\leadsto$  increase audit attention.

The forbidden interpretation is:

warning flag  $\leadsto$  automatic failure.

## H.8. Advisory invariant ledger entries

**Definition 1.31** (Advisory ledger entry). An *advisory ledger entry* records an advisory invariant together with:

- (i) its definition;
- (ii) the window, threshold, tower, or parameter region on which it was evaluated;
- (iii) the artifact reference supporting its computation;
- (iv) its advisory interpretation;
- (v) its non-verdict status.

**Remark 1.32** (Advisory ledger vs.  $\delta$ -ledger). An advisory ledger is not the same as the  $\delta$ -ledger. The  $\delta$ -ledger contributes to the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

An advisory ledger records inspection-supporting quantities. Unless explicitly converted into a declared budget component, an advisory indicator does not contribute to  $\Sigma\delta$ .

**Definition 1.33** (Budget-converted advisory quantity). An advisory quantity is *budget-converted* if a separate operational policy declares how it contributes to a budget component

$$\delta^{\text{adv}}$$

and how that component enters the total ledger

$$\Sigma\delta.$$

**Proposition 1.34** (Budget conversion requires explicit declaration). *An advisory quantity affects the Core budget condition only if it is explicitly budget-converted and included in the declared  $\delta$ -ledger.*

*Proof.* The Core budget condition depends only on declared budget components aggregated into  $\Sigma\delta$ . An advisory quantity outside the declared ledger is not part of  $\Sigma\delta$ . Therefore it affects the budget condition only after explicit budget conversion.  $\square$

## H.9. Advisory invariants and monitored Type IV

**Definition 1.35** (Type IV advisory proxy). A *Type IV advisory proxy* is an advisory invariant intended to correlate with possible monitored generic Type IV obstruction.

**Definition 1.36** (Actual Type IV diagnostic). The actual monitored generic Type IV diagnostic is computed from a declared tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

by taking the generic fiber dimensions of its kernel and cokernel defect profiles:

$$\mu_{i,W,\tau} = \text{gdim}(\ker \phi_{i,W,\tau}), \quad u_{i,W,\tau} = \text{gdim}(\text{coker } \phi_{i,W,\tau}),$$

and then summing over monitored degrees.

**Proposition 1.37** (Type IV proxy is not Type IV diagnosis). A *Type IV advisory proxy* does not determine whether

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \neq (0, 0).$$

*Proof.* Monitored Type IV diagnosis is defined by the generic fiber dimensions of kernel and cokernel defect profiles of the declared repaired tower comparison artifact

$$\phi_{i,W,\tau}.$$

A proxy may correlate with those quantities, but unless it computes them according to Appendix D using the declared comparison artifact, it is not the diagnosis itself.  $\square$

**Remark 1.38** (Correct use of Type IV proxies). A Type IV proxy may suggest where to compute  $\mu$  and  $u$ . It may not replace the computation of  $\mu$  and  $u$ .

**Remark 1.39** (Missing artifact cannot be repaired by a proxy). If Type IV diagnostics are invoked but the required tower comparison artifact is missing, an advisory proxy cannot repair that absence. The certificate remains incomplete with respect to the Type IV clause.

## H.10. Advisory invariants and stable bands

**Definition 1.40** (Advisory stable band). An *advisory stable band* is a threshold interval on which one or more advisory indicators appear stable according to a declared advisory policy.

**Definition 1.41** (Core stable band). A *Core stable band* is a threshold interval on which the monitored generic tower diagnostics vanish:

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

throughout the band, relative to declared tower comparison artifacts, as defined in Appendix D.

**Proposition 1.42** (Advisory stable band is not Core stable band). An *advisory stable band* is not a *Core stable band* unless the monitored generic diagnostics are explicitly verified throughout the band.

*Proof.* Advisory stability concerns advisory indicators. Core stability concerns the diagnostics

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}})$$

computed from declared tower comparison artifacts. These are distinct conditions. Therefore advisory stability alone cannot imply Core stability.  $\square$

**Remark 1.43** (Use of advisory bands). Advisory stable bands may be useful for proposing candidate Core stable bands. They must still be verified by the Core diagnostic machinery.

## H.11. Advisory invariants and certificate records

**Definition 1.44** (Advisory attachment to a certificate record). An *advisory attachment* is an optional section of a certificate record recording advisory invariants, spectral indicators, warning flags, navigation quantities, advisory stable bands, or Type IV proxies.

**Definition 1.45** (Advisory attachment status). An advisory attachment must be marked as one of:

- (i) non-verdict advisory data;
- (ii) proposer-side provenance;
- (iii) inspection aid;
- (iv) budget-converted data, if and only if an explicit conversion policy is declared;
- (v) verifier-side data, if and only if it has been re-entered through the Core schema.

**Proposition 1.46** (Advisory attachments are non-verdict data by default). *Advisory attachments do not affect the pass/reject status of a certificate record unless they are explicitly converted into verifier-side data.*

*Proof.* The pass/reject status is determined by the Core verifier-side data specified in Appendix G: declared objects, windows, thresholds, collapse policies, repaired evaluations, representatives and eligibility records when invoked, diagnostics, tower comparison artifacts, ledgers, gate verdicts, artifact references, and final status. Advisory attachments are optional non-verdict data by default. Therefore they do not affect pass/reject status unless explicitly re-entered through the verifier-side schema.  $\square$

**Remark 1.47** (Why include advisory attachments). Although advisory attachments do not determine the verdict, they can improve interpretability, debugging, prioritization, and reproducibility of exploratory decisions. They should be clearly marked as advisory.

## H.12. Optional quantitative collapse indicators

**Definition 1.48** (Residual bar-mass indicator). For a repaired windowed profile

$$\text{Eval}_{i,W,\tau}(F),$$

a *residual bar-mass indicator* is an advisory quantity measuring the total size, count, weighted length, or distributional mass of bars remaining after repaired windowed threshold deletion.

**Definition 1.49** (Collapse energy candidate). A *collapse energy candidate* is an advisory functional of the form

$$\mathcal{E}_{i,W,\tau}^{\text{adv}}(F)$$

assigned to repaired profiles or threshold sweeps, intended to quantify how far a profile appears from collapse. Unless separately promoted, it is advisory and not a Core gate.

**Proposition 1.50** (Collapse energy candidate is not B-Gate<sup>+</sup>). *A collapse energy candidate does not replace the B-Gate<sup>+</sup> conditions.*

*Proof.* B-Gate<sup>+</sup> requires exact repaired topological vanishing, eligible categorical vanishing when invoked, monitored generic Type IV cleanliness, declared artifacts, and budget pass. A scalar energy candidate may correlate with one or more of these, but it is not identical to their conjunction unless a separate Core theorem proves equivalence and the certificate manifest records the necessary verifier-side data.  $\square$

**Remark 1.51** (Future role of quantitative indicators). Quantitative collapse indicators may be useful in later quantitative extensions, such as a Collapse Energy calculus or Transport Theorem architecture. In the present appendix, they remain advisory.



### H.13. Summary of spectral advisory layer

**Theorem 1.52** (Appendix H advisory-invariant package). *Within the Technical Extension layer, the following package is available.*

- (i) *Spectral indicators and advisory invariants may be defined for windows, thresholds, repaired profiles, persistence objects, filtered objects, towers, comparison artifacts, or runs.*
- (ii) *Advisory invariants may guide attention, prioritization, warning, rechecking, or navigation.*
- (iii) *Advisory invariants do not update Core verdicts.*
- (iv) *Spectral stability does not imply B-Gate<sup>+</sup> pass.*
- (v) *Spectral instability does not imply Core reject.*
- (vi)  *$\Phi$  is a navigation quantity, not a certificate.*
- (vii) *Warning flags trigger inspection, not verdicts.*
- (viii) *Advisory quantities affect the budget condition only if explicitly budget-converted and included in the  $\delta$ -ledger.*
- (ix) *Type IV proxies do not replace  $\mu$ ,  $u$  diagnostics computed from declared tower comparison artifacts.*
- (x) *Advisory stable bands do not become Core stable bands without diagnostic verification.*
- (xi) *Advisory attachments to certificate records are non-verdict data by default.*
- (xii) *Collapse energy candidates and residual bar-mass indicators are advisory unless explicitly promoted through verifier-side data and theorem-level justification.*

*Proof.* Items (i)–(ii) follow from the definitions of advisory invariants and spectral indicators. Item (iii) is Proposition 1.7. Items (iv)–(v) are Propositions 1.16 and 1.17. Item (vi) is Proposition 1.25. Item (vii) is Proposition 1.29. Item (viii) is Proposition 1.34. Item (ix) is Proposition 1.37. Item (x) is Proposition 1.42. Item (xi) is Proposition 1.46. Item (xii) follows from Proposition 1.50.  $\square$

### H.14. Explicit exclusions

The following are not asserted in Appendix H.

- No spectral indicator is treated as a Core gate.
- No advisory invariant replaces B-Gate<sup>+</sup>.
- No advisory invariant replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No advisory invariant replaces the eligible Ext<sup>1</sup>-clause.
- No advisory invariant replaces monitored Type IV diagnostics.

- No advisory invariant replaces the tower comparison artifact

$$\phi_{i,w,\tau}.$$

- No advisory invariant replaces the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

- No value of  $\Phi$  is treated as a proof.
- No potential landscape is treated as a certificate.
- No warning flag is treated as automatic reject.
- No advisory stable band is treated as a Core stable band without diagnostic verification.
- No Type IV proxy is treated as the actual computation of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No collapse energy candidate is treated as a Core theorem or Core gate.
- No AI-generated ranking, confidence, or recommendation is treated as a Core verdict.
- No Search-layer artifact is promoted to theorem status.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved here.
- No global exactness of  $T_\tau^{\text{bar}}$  is assumed.
- No global strict filtered lift  $C_\tau$  is assumed.

### H.15. Provenance and status

The material in this appendix depends on:

- (i) the repaired windowed evaluation discipline of Chapters 1–2 and Appendix A;
- (ii) the representative discipline of Appendix B;
- (iii) the monitored Type IV diagnostic calculus of Appendix D;
- (iv) the Core sovereignty doctrine of Chapter 6;
- (v) the non-claim discipline of Chapter 7;
- (vi) the manifest distinction between verifier-side and proposer-side data in Appendix G.

Its role is to allow useful technical indicators without allowing them to contaminate Core verdicts.

**Status labels for Appendix H.** *Technical extension definitions:* Definitions 1.4, 1.5, 1.6, 1.9, 1.10, 1.11, 1.12, 1.15, 1.19, 1.23, 1.24, 1.27, 1.28, 1.31, 1.33, 1.35, 1.36, 1.40, 1.41, 1.44, 1.45, 1.48, 1.49.

*Technical extension propositions/theorems:* Propositions 1.7, 1.16, 1.17, 1.20, 1.25, 1.29, 1.34, 1.37, 1.42, 1.46, 1.50, Theorem 1.52.

*Operational cautions:* Remarks 1.1, 1.2, 1.3, 1.8, 1.13, 1.14, 1.18, 1.21, 1.22, 1.26, 1.30, 1.32, 1.38, 1.39, 1.43, 1.47, 1.51.

*Explicit exclusions:* Section H.14.

## 2 Appendix I: Grid-to-Continuum and Discretization Stability

### I.1. Scope and non-Core status

This appendix records optional grid-to-continuum and discretization-stability principles used when Core-relevant persistence data are approximated from finite grids, samples, meshes, numerical fields, finite truncations, or discretized observables.

The material here belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Discretized evidence may support a certificate only through declared error budgets and Core re-entry.

In particular, a grid computation, numerical approximation, finite mesh, sampled observable, computed barcode, or discretized tower diagnostic does not by itself determine Core pass/reject. It must be incorporated into the audit discipline through:

$$\delta^{\text{disc}}, \quad \Sigma\delta < \text{gap}_\tau, \quad \text{declared artifact references}, \quad \text{and repaired Core gate verification.}$$

All Core-relevant persistence data are evaluated through the repaired windowed profile

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

**Remark 2.1** (Technical extension status). This appendix does not claim that every discrete approximation converges to a valid continuous object. It only specifies the conditions under which discretization error may be declared, bounded, recorded, and entered into the  $\delta$ -ledger.

**Remark 2.2** (Relation to Core sovereignty). Core sovereignty remains intact. A discretized computation may propose, approximate, or support a certificate record, but it does not replace B-Gate<sup>+</sup>, monitored Type IV diagnostics, tower comparison artifacts, or the budget comparison.

**Remark 2.3** (Relation to Appendix G). If a certificate record relies on discretized evidence, the discretization scale, realization route, error bound, adjusted budget, and artifact references must be recorded in the manifest schema of Appendix G. Otherwise the certificate is not audit-readable.

### I.2. Discretized objects and continuum targets

**Definition 2.4** (Continuum target). A *continuum target* is a declared mathematical object

$$X$$

from which Core-relevant filtered or persistence data are intended to be derived. Examples include a continuous function, filtered space, analytic field, geometric object, categorical object, continuous-time trajectory, or external-domain object under an interface realization.

**Definition 2.5** (Discretized object). A *discretized object* associated to a continuum target  $X$  is a finite or locally finite approximation

$$X_h$$

indexed by a discretization scale  $h > 0$ , such as a grid, mesh, point sample, finite truncation, numerical approximation, finite-mode projection, or discrete surrogate.

**Definition 2.6** (Discretization family). A *discretization family* is a family

$$\{X_h\}_{h \in H}$$

of discretized objects indexed by a directed set of scales  $H$ , usually with  $h \rightarrow 0$  interpreted as refinement.

**Definition 2.7** (Core-relevant realization of a discretized object). A discretized object  $X_h$  is *Core-relevant* if it is equipped with a declared realization

$$\mathfrak{R}_h(X_h) \in \text{FiltCh}(( )k) \quad \text{or} \quad \mathfrak{R}_h(X_h) \in \text{Pers}_k^{\text{cons}},$$

so that the repaired Core-relevant data can be evaluated as

$$\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)).$$

**Definition 2.8** (Continuum realization). A continuum target  $X$  has a *continuum realization* if a declared route

$$\mathfrak{R}(X) \in \text{FiltCh}(( )k) \quad \text{or} \quad \mathfrak{R}(X) \in \text{Pers}_k^{\text{cons}}$$

is supplied and its repaired windowed profiles

$$\text{Eval}_{i,W,\tau}(\mathfrak{R}(X))$$

are defined in the constructible monitored regime.

**Remark 2.9** (No raw numerical substitution). A numerical grid, mesh, sample, or solver output does not directly substitute for a Core object. It must first be realized as a filtered or persistence object through a declared realization map. This is the discretization analogue of the interface discipline in Chapter 8.

**Remark 2.10** (Constructibility requirement). The Core uses constructible one-parameter persistence over  $k$ . A discretized or continuum realization outside this regime is not Core-readable unless a constructible approximation, truncation, or replacement is declared and its error is ledgered.

### I.3. Discretization error

**Definition 2.11** (Discretization error). Let  $X$  be a continuum target and  $X_h$  a discretized object. A *discretization error* is a declared bound

$$\delta_h^{\text{disc}}$$

measuring the discrepancy between the repaired Core-relevant data derived from  $X_h$  and the corresponding repaired data intended for  $X$ .

**Definition 2.12** (Persistence-level discretization bound). A discretization family satisfies a *persistence-level discretization bound* on  $(W, \tau, i)$  if there exists

$$\delta_h^{\text{disc}}(W, \tau, i) \geq 0$$

such that

$$d_{\text{int}}(\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,W,\tau}(\mathfrak{R}(X))) \leq \delta_h^{\text{disc}}(W, \tau, i).$$

**Definition 2.13** (Pre-collapse discretization bound). A discretization family satisfies a *pre-collapse discretization bound* on  $(W, i)$  if there exists

$$\epsilon_h(W, i) \geq 0$$

such that

$$d_{\text{int}}(W_W \mathbf{P}_i(\mathfrak{R}_h(X_h)), W_W \mathbf{P}_i(\mathfrak{R}(X))) \leq \epsilon_h(W, i).$$

When the barcode-level threshold deletion is 1-Lipschitz on the declared barcode metric regime, this yields the repaired bound

$$d_{\text{int}}(\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,W,\tau}(\mathfrak{R}(X))) \leq \epsilon_h(W, i).$$

**Definition 2.14** (Declared discretization ledger entry). A *declared discretization ledger entry* is a record containing:

- (i) the discretization scale  $h$ ;
- (ii) the discretized object  $X_h$ ;
- (iii) the continuum target  $X$ , if used;
- (iv) the realization maps  $\mathfrak{R}_h$  and  $\mathfrak{R}$ , when both are present;
- (v) the bound  $\delta_h^{\text{disc}}$ ;
- (vi) the window  $W$ , threshold  $\tau$ , and monitored degree  $i$ ;
- (vii) the repaired profiles being compared;
- (viii) artifact references supporting the bound.

**Remark 2.15** (Discretization error belongs to the ledger). Discretization error is not a side note. If a certificate uses discretized data, the corresponding error must enter the  $\delta$ -ledger as a declared component:

$$\delta^{\text{disc}} \subset \Sigma\delta.$$

#### I.4. Stability transfer through repaired windowed evaluation

**Proposition 2.16** (Window restriction does not amplify discretization error). *If*

$$d_{\text{int}}(\mathbf{P}_i(\mathfrak{R}_h(X_h)), \mathbf{P}_i(\mathfrak{R}(X))) \leq \delta_h^{\text{disc}},$$

*then, under the declared window-restriction stability regime,*

$$d_{\text{int}}(W_W \mathbf{P}_i(\mathfrak{R}_h(X_h)), W_W \mathbf{P}_i(\mathfrak{R}(X))) \leq \delta_h^{\text{disc}}.$$

*Proof.* Window restriction is 1-Lipschitz for the interleaving distance in the declared constructible persistence regime. Therefore restriction to  $W$  cannot increase the discrepancy.  $\square$

**Proposition 2.17** (Barcode-level threshold deletion does not amplify discretization error). *Assume a discretized object and continuum target satisfy*

$$d_{\text{int}}(W_W \mathbf{P}_i(\mathfrak{R}_h(X_h)), W_W \mathbf{P}_i(\mathfrak{R}(X))) \leq \delta_h^{\text{disc}}.$$

*Then, under the declared barcode-level 1-Lipschitz stability of  $T_\tau^{\text{bar}}$ ,*

$$d_{\text{int}}(\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,W,\tau}(\mathfrak{R}(X))) \leq \delta_h^{\text{disc}}.$$

*Proof.* The repaired evaluation is

$$\text{Eval}_{i,W,\tau}(-) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(-)).$$

By the barcode-level stability of  $T_\tau^{\text{bar}}$ , applying threshold deletion to the two windowed persistence profiles cannot increase their interleaving distance.  $\square$

**Theorem 2.18** (Discretization stability under repaired Core window evaluation). *If the raw persistence-level discretization error is bounded by*

$$\delta_h^{\text{disc}},$$

*then the repaired Core windowed profiles satisfy*

$$d_{\text{int}}(\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,W,\tau}(\mathfrak{R}(X))) \leq \delta_h^{\text{disc}}$$

*under the declared window-restriction and barcode-level threshold-deletion stability assumptions.*

*Proof.* Apply Proposition 2.16 followed by Proposition 2.17.  $\square$

**Remark 2.19** (Why this matters). This theorem is the main reason discretization error can be safely ledgered before repaired after-collapse gate evaluation. The repaired windowed evaluation does not amplify the declared interleaving error under the stated stability assumptions.

**Remark 2.20** (No exact collapse functor is used). The stability statement above uses barcode-level non-amplification of  $T_\tau^{\text{bar}}$ . It does not use a global exact collapse functor, a Serre localization, or an exact reflector on all constructible persistence modules.

## I.5. Grid-to-continuum convergence

**Definition 2.21** (Grid-to-continuum convergence). A discretization family  $\{X_h\}$  converges to a continuum target  $X$  in the repaired Core-relevant persistence sense if, for every declared window  $W$ , threshold  $\tau$ , and monitored degree  $i$ ,

$$d_{\text{int}}(\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,W,\tau}(\mathfrak{R}(X))) \rightarrow 0$$

as  $h \rightarrow 0$ .

**Definition 2.22** (Convergence rate). A function

$$\rho(h) \rightarrow 0 \quad (h \rightarrow 0)$$

is a *repaired Core-relevant convergence rate* if

$$d_{\text{int}}(\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)), \text{Eval}_{i,W,\tau}(\mathfrak{R}(X))) \leq \rho(h)$$

for the declared windows, thresholds, and monitored degrees.

**Remark 2.23** (Convergence is conditional). Grid-to-continuum convergence is not automatic. It must be proved, assumed, or declared as an interface hypothesis with artifact support. A numerical refinement sequence alone does not establish convergence.

**Proposition 2.24** (Convergence implies vanishing discretization ledger). *If a discretization family has convergence rate  $\rho(h)$ , then one may declare*

$$\delta_h^{\text{disc}} := \rho(h),$$

and

$$\delta_h^{\text{disc}} \rightarrow 0 \quad (h \rightarrow 0).$$

*Proof.* By definition of convergence rate,  $\rho(h)$  bounds the repaired Core-relevant persistence discrepancy. Thus it is an admissible discretization ledger component. Since  $\rho(h) \rightarrow 0$ , the declared discretization error vanishes as  $h \rightarrow 0$ .  $\square$

## I.6. Discretized B-Gate<sup>+</sup> and audit discipline

**Definition 2.25** (Discretized B-Gate<sup>+</sup> record). A *discretized B-Gate<sup>+</sup> record* is a certificate record in which one or more gate-relevant quantities are computed from discretized objects  $X_h$ . Such a record must include:

- (i) the discretization scale  $h$ ;
- (ii) the realized discretized object  $\mathfrak{R}_h(X_h)$ ;
- (iii) the repaired evaluation records

$$\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h));$$

- (iv) the discretization ledger component

$$\delta_h^{\text{disc}};$$

- (v) artifact references supporting the discretized computation and the error bound.

**Definition 2.26** (Discretization-adjusted budget). The *discretization-adjusted budget* is

$$\Sigma\delta_{\text{adj}} := \Sigma\delta_{\text{base}} \oplus \delta_h^{\text{disc}},$$

where  $\Sigma\delta_{\text{base}}$  is the total budget excluding discretization error.

**Proposition 2.27** (Discretized evidence must pass the adjusted budget). *A certificate record using discretized evidence can pass Audit Mode only if*

$$\Sigma\delta_{\text{adj}} < \text{gap}_{\tau}.$$

*Proof.* By Chapter 6, Audit Mode requires the total declared budget to satisfy

$$\Sigma\delta < \text{gap}_{\tau}.$$

If discretized evidence is used, its declared discretization error is part of the total budget. Thus the adjusted budget must satisfy the strict inequality.  $\square$

**Remark 2.28** (Discretized pass is still Core pass). A pass based on discretized evidence is a Core pass only if all ordinary Core conditions pass after accounting for discretization error. The discretization itself does not create a separate acceptance rule.

**Remark 2.29** (Discretized topological pass). For discretized evidence, the repaired topological clause is still read as

$$\text{Eval}_{1,W,\tau}(\mathfrak{R}_h(X_h)) = 0$$

together with an error ledger sufficient to support the intended certificate. A raw computed barcode image or table is not itself the gate verdict unless it is recorded as a repaired evaluation artifact.

## I.7. Resolution thresholds and collapse thresholds

**Definition 2.30** (Grid resolution). The *grid resolution*  $h$  is the declared scale at which a discretized object  $X_h$  approximates the continuum target  $X$ .

**Definition 2.31** (Resolution-safe collapse). A collapse threshold  $\tau$  is *resolution-safe* at grid resolution  $h$  if the adjusted budget including discretization error satisfies

$$\Sigma\delta_{\text{base}} \oplus \delta_h^{\text{disc}} < \text{gap}_\tau.$$

**Remark 2.32** (Resolution and threshold must be coordinated). A fine collapse threshold  $\tau$  may be meaningless if the grid resolution error is too large relative to  $\text{gap}_\tau$ . Therefore  $\tau$  and  $h$  must be coordinated through the ledger.

**Proposition 2.33** (Refinement can restore budget admissibility). *Suppose*

$$\delta_h^{\text{disc}} \rightarrow 0$$

*as  $h \rightarrow 0$ , and suppose the non-discretization budget satisfies*

$$\Sigma\delta_{\text{base}} < \text{gap}_\tau.$$

*Then for sufficiently small  $h$ ,*

$$\Sigma\delta_{\text{base}} \oplus \delta_h^{\text{disc}} < \text{gap}_\tau$$

*provided the budget aggregation law is continuous with respect to decreasing discretization error.*

*Proof.* Since

$$\delta_h^{\text{disc}} \rightarrow 0,$$

and the base budget is strictly below the margin, there is positive slack. Continuity of the aggregation law implies that, for sufficiently small  $h$ , the discretization contribution remains within that slack. Thus the adjusted budget is admissible.  $\square$

## I.8. Sampling and aliasing warnings

**Definition 2.34** (Sampling error). A *sampling error* is a discretization error arising from observing only finitely many points, cells, modes, events, measurements, or sampled values of a continuum target. It is recorded as a component

$$\delta_h^{\text{samp}}$$

of the discretization ledger.

**Definition 2.35** (Aliasing risk). An *aliasing risk* is an advisory warning that unresolved features at scales below the discretization resolution  $h$  may be misrepresented as features at larger scales.

**Remark 2.36** (Aliasing is advisory unless budgeted). Aliasing risk is advisory unless converted into a declared budget component. If it is used in a certificate, it must be represented by an explicit ledger contribution or by a documented refinement policy.

**Definition 2.37** (Sampling adequacy condition). A discretized run satisfies *sampling adequacy* if the declared sampling error

$$\delta_h^{\text{samp}}$$

is included in the total budget and the adjusted budget passes:

$$\Sigma\delta_{\text{base}} \oplus \delta_h^{\text{samp}} < \text{gap}_\tau.$$



**Proposition 2.38** (Sampling adequacy is necessary for sampled evidence). *If a certificate record relies on sampled evidence, then sampling adequacy is necessary for audited acceptance.*

*Proof.* Sampled evidence introduces sampling error. By the ledger discipline, all admissible deviations must be included in  $\Sigma\delta$ . Therefore the sampling contribution must be budgeted and the adjusted budget must pass.  $\square$

## I.9. Grid-to-continuum and monitored Type IV

**Definition 2.39** (Discretized tower). *A discretized tower is a tower*

$$\{F_{n,h}\}_{n \in \mathbb{N}}$$

depending on a discretization scale  $h$ , intended to approximate a continuum tower

$$\{F_n\}_{n \in \mathbb{N}}.$$

**Definition 2.40** (Discretized tower comparison artifact). *A discretized tower comparison artifact is a declared comparison*

$$\phi_{i,W,\tau}^{(h)} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet,h}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty,h})$$

constructed from the discretized tower at scale  $h$ .

**Definition 2.41** (Discretized tower diagnostics). *The discretized tower diagnostics are*

$$\mu_{\text{Collapse}}^{(h)}, \quad u_{\text{Collapse}}^{(h)},$$

computed from the kernel and cokernel defect profiles of the discretized tower comparison artifacts

$$\phi_{i,W,\tau}^{(h)}.$$

**Remark 2.42** (Discretized diagnostics are approximations). *The quantities*

$$\mu_{\text{Collapse}}^{(h)}, \quad u_{\text{Collapse}}^{(h)}$$

are not automatically equal to the continuum diagnostics. They are discretized diagnostics and must be related to the continuum diagnostics by a declared stability or convergence statement.

**Definition 2.43** (Continuum tower comparison artifact). *A continuum tower comparison artifact is a declared comparison*

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty})$$

for the continuum tower.

**Definition 2.44** (Type IV diagnostic convergence). *A discretized tower has Type IV diagnostic convergence if, for sufficiently small  $h$ , the discretized defect profiles of*

$$\phi_{i,W,\tau}^{(h)}$$

converge to the continuum defect profiles of

$$\phi_{i,W,\tau}$$

in a declared persistence metric, and their generic fiber dimensions agree on the monitored degrees.

**Proposition 2.45** (Discretized zero Type IV does not imply continuum zero Type IV). *The condition*

$$(\mu_{\text{Collapse}}^{(h)}, u_{\text{Collapse}}^{(h)}) = (0, 0)$$

*at a fixed discretization scale  $h$  does not imply*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

*for the continuum tower.*

*Proof.* A fixed discretization may fail to resolve continuum-level generic kernel or cokernel defects. Without a convergence or stability theorem connecting the discretized defect profiles to the continuum defect profiles, no implication from discretized zero diagnostics to continuum zero diagnostics is valid.  $\square$

**Proposition 2.46** (Diagnostic convergence licenses transfer). *If Type IV diagnostic convergence holds and, for sufficiently small  $h$ ,*

$$(\mu_{\text{Collapse}}^{(h)}, u_{\text{Collapse}}^{(h)}) = (0, 0),$$

*then the continuum tower is Type IV-free in the monitored generic sense.*

*Proof.* By Type IV diagnostic convergence, the discretized defect profiles converge to the continuum defect profiles and their generic fiber dimensions agree for sufficiently small  $h$ . Thus the vanishing of discretized generic kernel and cokernel defects transfers to the continuum diagnostics.  $\square$

**Remark 2.47** (Transfer requires declared convergence). The transfer in Proposition 2.46 is not automatic. It depends entirely on the declared diagnostic convergence hypothesis.

**Remark 2.48** (Missing continuum artifact). A discretized tower comparison artifact does not replace the continuum tower comparison artifact unless a declared convergence or reconstruction theorem supplies the continuum artifact from the discretized family.

## I.10. Discretization artifacts

**Definition 2.49** (Discretization artifact). A *discretization artifact* is an artifact supporting a discretized certificate record. Examples include:

- (i) grid definitions;
- (ii) mesh files;
- (iii) sampling rules;
- (iv) finite truncation parameters;
- (v) numerical field data;
- (vi) discrete filtered complexes;
- (vii) computed barcodes;
- (viii) repaired evaluation records;
- (ix) discretization error estimates;
- (x) refinement logs;

- (xi) convergence certificates;
- (xii) discretized tower comparison artifacts.

**Definition 2.50** (Discretization manifest entry). *A discretization manifest entry records:*

- (i) the discretization scale  $h$ ;
- (ii) the discretization method;
- (iii) the realized object  $\mathfrak{R}_h(X_h)$ ;
- (iv) the repaired evaluation record
 
$$\text{Eval}_{i,w,\tau}(\mathfrak{R}_h(X_h));$$
- (v) the discretization artifact references;
- (vi) the declared error bound  $\delta_h^{\text{disc}}$ ;
- (vii) how that error enters the ledger.

**Proposition 2.51** (Discretization artifacts are required for reproducibility). *A certificate record relying on discretized evidence is not audit-readable unless it includes discretization artifact references.*

*Proof.* Without artifact references, an independent verifier cannot reconstruct the discretized object, the realization map, the repaired evaluation records, the computed persistence data, or the declared discretization error. Thus the certificate record fails the audit-readable schema of Appendix G.  $\square$

## I.11. Grid refinement policy

**Definition 2.52** (Grid refinement policy). *A grid refinement policy specifies how a discretization scale  $h$  is refined, how discretized objects  $X_h$  are updated, and how the corresponding discretization error bounds are recomputed.*

**Definition 2.53** (Refinement-stable advisory pattern). *An advisory pattern is refinement-stable if it persists across a declared sequence of refinements*

$$h_1 > h_2 > \dots \rightarrow 0.$$

**Remark 2.54** (Advisory refinement stability is not Core convergence). *Refinement-stable advisory behavior does not prove Core convergence. It may suggest that convergence should be investigated, but a repaired Core-relevant convergence bound or diagnostic transfer theorem is still required.*

**Proposition 2.55** (Refinement policy does not replace error bounds). *A grid refinement policy does not replace the need for declared discretization error bounds.*

*Proof.* A refinement policy describes how approximations are generated. It does not, by itself, quantify the discrepancy between the discretized and continuum repaired Core-relevant data. The  $\delta$ -ledger requires an explicit error bound.  $\square$

## I.12. Summary of grid-to-continuum extension

**Theorem 2.56** (Appendix I discretization-stability package). *Within the Technical Extension layer, the following package is available.*

(i) *Discretized objects may support Core certification only through declared realizations.*

(ii) *Core-relevant discretized data are evaluated through*

$$\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(\mathfrak{R}_h(X_h))).$$

(iii) *Discretization error must be recorded as a ledger component*

$$\delta^{\text{disc}}.$$

(iv) *Window restriction and barcode-level threshold deletion do not amplify declared interleaving discretization error under the stated stability assumptions.*

(v) *Grid-to-continuum convergence must be declared, proved, or assumed as an interface hypothesis; it is not automatic.*

(vi) *A discretized B-Gate<sup>+</sup> record must pass the adjusted budget condition*

$$\Sigma \delta_{\text{base}} \oplus \delta_h^{\text{disc}} < \text{gap}_\tau.$$

(vii) *Resolution  $h$  and collapse threshold  $\tau$  must be coordinated through the ledger.*

(viii) *Sampling error is a discretization error and must be budgeted when sampled evidence is used.*

(ix) *Discretized Type IV diagnostics do not automatically imply continuum Type IV diagnostics.*

(x) *Transfer from discretized diagnostics to continuum diagnostics requires declared diagnostic convergence.*

(xi) *Discretization artifact references are required for audit readability.*

(xii) *Refinement policies do not replace error bounds.*

(xiii) *No global exact collapse functor is used in this discretization-stability package.*

*Proof.* Items (i)–(iii) follow from the realization and ledger definitions. Item (iv) follows from Propositions 2.16 and 2.17. Item (v) is the convergence caveat of Definition 2.21. Item (vi) is Proposition 2.27. Item (vii) follows from the resolution-safe collapse condition. Item (viii) follows from sampling adequacy. Items (ix)–(x) follow from Propositions 2.45 and 2.46. Item (xi) is Proposition 2.51. Item (xii) is Proposition 2.55. Item (xiii) follows from Remark 2.20.  $\square$

## I.13. Explicit exclusions

The following are not asserted in Appendix I.

- No grid computation is treated as a Core proof by itself.
- No discretized barcode is treated as the continuum barcode without a declared error or convergence statement.

- No finite mesh is treated as an exact continuum object.
- No discretized B-Gate<sup>+</sup> pass is accepted without adding discretization error to the ledger.
- No grid-to-continuum convergence is assumed automatically.
- No refinement pattern is treated as proof of convergence.
- No discretized zero Type IV diagnostic is treated as continuum zero Type IV diagnostic without diagnostic convergence.
- No discretized tower comparison artifact replaces the continuum tower comparison artifact without a declared convergence or reconstruction theorem.
- No sampling adequacy is assumed without a declared sampling error component.
- No aliasing warning is treated as a Core verdict.
- No numerical stability observation replaces B-Gate<sup>+</sup>, monitored diagnostics, tower comparison artifacts, or budget comparison.
- No raw numerical field replaces the repaired evaluation record

$$\text{Eval}_{i,w,\tau}(F).$$

- No global exactness of  $T_\tau^{\text{bar}}$  is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift  $C_\tau$  is assumed.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved here.

#### **I.14. Provenance and status**

The material in this appendix depends on:

- (i) the repaired barcode-level collapse and 1-Lipschitz stability discipline of Appendix A;
- (ii) the admissible representative conventions of Appendix B;
- (iii) the monitored Type IV diagnostic calculus of Appendix D;
- (iv) the ledger and audit discipline of Chapter 6;
- (v) the manifest requirements of Appendix G;
- (vi) the advisory-invariant discipline of Appendix H.

Its role is to allow discretized evidence to support Core certification only through explicit error accounting and audit re-entry.

**Status labels for Appendix I.** *Technical extension definitions:* Definitions 2.4, 2.5, 2.6, 2.7, 2.8, 2.11, 2.12, 2.13, 2.14, 2.21, 2.22, 2.25, 2.26, 2.30, 2.31, 2.34, 2.35, 2.37, 2.39, 2.40, 2.41, 2.43, 2.44, 2.49, 2.50, 2.52, 2.53.

*Technical extension propositions/theorems:* Propositions 2.16, 2.17, Theorem 2.18, Propositions 2.24, 2.27, 2.33, 2.38, 2.45, 2.46, 2.51, 2.55, Theorem 2.56.

*Operational cautions:* Remarks 2.1, 2.2, 2.3, 2.9, 2.10, 2.15, 2.19, 2.20, 2.23, 2.28, 2.29, 2.32, 2.36, 2.42, 2.47, 2.48, 2.54.

*Explicit exclusions:* Section I.13.

### 3 Appendix J: Betti Integral and Measurement-Level Stability

#### J.1. Scope and non-Core status

This appendix records optional measurement-level invariants derived from Betti curves, barcode summaries, repaired persistence profiles, and related aggregate quantities. Its central example is the *Betti integral*.

The material here belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Measurement-level invariants may summarize evidence, but they do not determine Core verdicts.

In particular, no Betti integral, Betti curve, aggregate persistence score, measurement profile, visual plot, or numerical summary may replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_\tau.$$

All Core-relevant measurement quantities in this appendix are computed from the repaired windowed collapsed profile

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)),$$

unless explicitly declared as raw, advisory, or pre-collapse data.

**Remark 3.1** (Technical extension status). The quantities introduced here are measurement-level summaries. They may help compare windows, identify anomalous regions, summarize persistent mass, or estimate stability under observation. They are not Core gates and do not certify a proof object by themselves.

**Remark 3.2** (Relation to advisory invariants). Appendix H treated spectral indicators and advisory invariants. The present appendix treats Betti-integral and measurement-level quantities. The same sovereignty rule applies: measurement summaries do not update Core pass/reject verdicts unless they are explicitly converted into ledger components and re-entered through the Core audit discipline.

**Remark 3.3** (Relation to discretization). Appendix I handles grid-to-continuum and discretization error. If a Betti curve or Betti integral is computed from discretized data, then both the discretization error and the measurement error must be represented in the declared ledger.

## J.2. Repaired Betti curves

**Definition 3.4** (Windowed repaired Betti number). Let  $F$  be a filtered object,  $W = [u, u')$  a right-open window,  $\tau > 0$  a collapse threshold, and  $i$  a monitored degree. The *windowed repaired Betti number* at parameter  $t \in W$  is

$$\beta_{i,W,\tau}^F(t) := \dim_k \left( \text{Eval}_{i,W,\tau}(F)(t) \right).$$

Equivalently,

$$\beta_{i,W,\tau}^F(t) = \dim_k \left( T_{\tau}^{\text{bar}}(\mathbb{W}_W \mathbf{P}_i(F)) \right)(t).$$

**Definition 3.5** (Repaired Betti curve). The function

$$\beta_{i,W,\tau}^F : W \rightarrow \mathbb{N}$$

is called the *windowed repaired Betti curve* of  $F$  in degree  $i$ , window  $W$ , and threshold  $\tau$ .

**Remark 3.6** (After-collapse convention). Betti curves used for Core-adjacent measurement in this appendix are evaluated after the repaired Core window-evaluation order:

$$F \mapsto \mathbf{P}_i(F) \mapsto \mathbb{W}_W \mathbf{P}_i(F) \mapsto T_{\tau}^{\text{bar}}(\mathbb{W}_W \mathbf{P}_i(F)) = \text{Eval}_{i,W,\tau}(F).$$

This avoids silently using raw pre-collapse Betti curves as if they were Core-level objects.

**Definition 3.7** (Raw Betti curve). A *raw Betti curve* is a Betti curve computed before applying the declared repaired windowed collapse evaluation. Raw Betti curves are measurement-level or advisory data unless they are explicitly converted into repaired Core evaluation data.

**Remark 3.8** (Raw Betti curves are not Core data). Raw Betti curves may be useful for exploratory analysis. However, they do not determine B-Gate<sup>+</sup>, Type IV diagnostics, or audited acceptance. Core-relevant Betti measurements must be tied to

$$\text{Eval}_{i,W,\tau}(F).$$

## J.3. Betti integral

**Definition 3.9** (Betti integral). Let  $F$ ,  $W$ ,  $\tau$ , and  $i$  be as above. The *Betti integral* is the measurement-level quantity

$$\mathcal{B}_{i,W,\tau}(F) := \int_W \beta_{i,W,\tau}^F(t) dt,$$

whenever this integral is finite.

**Definition 3.10** (Discrete Betti integral). Given a finite sampling

$$u = t_0 < t_1 < \dots < t_m = u'$$

of  $W = [u, u')$ , the *discrete Betti integral* is

$$\mathcal{B}_{i,W,\tau}^{\text{disc}}(F) := \sum_{j=0}^{m-1} \beta_{i,W,\tau}^F(t_j) (t_{j+1} - t_j).$$

**Definition 3.11** (Normalized Betti integral). If  $W = [u, u')$  has finite positive length, the *normalized Betti integral* is

$$\overline{\mathcal{B}}_{i,W,\tau}(F) := \frac{1}{u' - u} \mathcal{B}_{i,W,\tau}(F).$$

**Remark 3.12** (Meaning of the Betti integral). The Betti integral measures the total repaired Betti mass across the window. It is an aggregate measurement. It is not equivalent to pointwise vanishing unless constructibility, nonnegativity, and endpoint conditions are explicitly used.

**Remark 3.13** (Measurement, not verdict). The Betti integral may summarize how much repaired persistence mass remains in a window. It does not, by itself, decide the Core gate. The Core gate is still evaluated through repaired profile vanishing, monitored Type IV diagnostics, artifact requirements, and budget comparison.

#### J.4. Zero Betti integral and repaired vanishing

**Proposition 3.14** (Zero Betti integral implies almost-everywhere vanishing). *Assume  $\beta_{i,W,\tau}^F$  is a constructible nonnegative integer-valued function on  $W$ . If*

$$\mathcal{B}_{i,W,\tau}(F) = 0,$$

*then*

$$\beta_{i,W,\tau}^F(t) = 0$$

*for almost every  $t \in W$ . If, in addition, the Betti curve is right-continuous and constructible with no positive-measure ambiguity at endpoints, then it vanishes on every nontrivial subinterval of  $W$ .*

*Proof.* The Betti curve is nonnegative. If its integral is zero, then it must vanish almost everywhere. Constructibility implies the curve is piecewise constant with finitely many changes on bounded windows. If it were positive on any nontrivial subinterval, the integral would be positive. Thus it vanishes on every nontrivial interval component away from endpoint conventions.  $\square$

**Remark 3.15** (Zero integral is not automatically B-Gate<sup>+</sup>). Even when a Betti integral vanishes in degree 1, this is not by itself a full B-Gate<sup>+</sup> pass. The eligible Ext<sup>1</sup>-clause, monitored Type IV diagnostics, required tower comparison artifacts, and budget condition must still be checked.

**Definition 3.16** (Degree-1 Betti integral). The degree-1 Betti integral on  $(F; W, \tau)$  is

$$\mathcal{B}_{1,W,\tau}(F) := \int_W \beta_{1,W,\tau}^F(t) dt.$$

**Proposition 3.17** (Degree-1 Betti integral as a repaired vanishing witness). *Assume the repaired degree-1 Betti curve is constructible and nonnegative. If*

$$\mathcal{B}_{1,W,\tau}(F) = 0$$

*and the window convention excludes endpoint-only residual ambiguity, then*

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

*Consequently, for any admissible representative  $C_{\tau,W}F$ ,*

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

*Proof.* By Proposition 3.14, the repaired degree-1 Betti curve vanishes on every nontrivial subinterval of  $W$ . Thus the repaired degree-1 persistence profile has no residual bar with positive length in  $W$ . Under the endpoint convention excluding endpoint-only residual ambiguity, this gives

$$\text{Eval}_{1,W,\tau}(F) = 0.$$



If  $C_{\tau,W}F$  is an admissible representative, then

$$\mathbf{P}_1(C_{\tau,W}F) \cong \text{Eval}_{1,W,\tau}(F),$$

so

$$\text{PH}_1(C_{\tau,W}F) = 0.$$

□

**Remark 3.18** (Witness, not replacement). The degree-1 Betti integral may serve as a measurement-level witness for the repaired topological clause of B-Gate<sup>+</sup> under explicit constructibility and endpoint hypotheses. It does not replace the full gate. It also does not prove the Ext<sup>1</sup>-clause except through the eligible one-way bridge of Chapter 3 and Appendix C.

**Proposition 3.19** (Positive Betti integral indicates residual repaired mass). *If*

$$\mathcal{B}_{1,W,\tau}(F) > 0,$$

*then the repaired degree-1 Betti curve is nonzero on a subset of positive measure. In particular, the repaired topological clause*

$$\text{Eval}_{1,W,\tau}(F) = 0$$

*fails unless the positivity is entirely attributable to declared measurement or discretization error.*

*Proof.* A positive integral of a nonnegative constructible function implies that the function is positive on a subset of positive measure. Thus residual repaired degree-1 mass is present. If no measurement or discretization error explains this positivity, the repaired topological vanishing condition fails. □

**Remark 3.20** (Error-qualified positivity). Observed positivity in a measured or discretized Betti integral must be interpreted through the ledger. A small positive observed value may be compatible with true repaired vanishing only if the measurement and discretization error bounds justify that interpretation.

## J.5. Measurement error

**Definition 3.21** (Measurement error). A *measurement error* is a declared bound

$$\delta^{\text{meas}}$$

on the discrepancy between the measured value of a quantity and the ideal repaired value used in Core-adjacent evaluation.

**Definition 3.22** (Betti-integral measurement error). Let

$$\widehat{\mathcal{B}}_{i,W,\tau}(F)$$

be a measured or computed approximation to

$$\mathcal{B}_{i,W,\tau}(F).$$

A *Betti-integral measurement error* is a bound

$$\left| \widehat{\mathcal{B}}_{i,W,\tau}(F) - \mathcal{B}_{i,W,\tau}(F) \right| \leq \delta_{\mathcal{B}}^{\text{meas}}.$$

**Definition 3.23** (Measured vanishing certificate). A *measured vanishing certificate* for the Betti integral consists of:

(i) a measured value

$$\widehat{\mathcal{B}}_{1,W,\tau}(F);$$

(ii) an error bound

$$\delta_{\mathcal{B}}^{\text{meas}};$$

(iii) a declared rule justifying that the true repaired Betti integral is zero or below a required vanishing threshold;

(iv) artifact references supporting the measurement and the error bound.

**Definition 3.24** (Measurement ledger entry). A *measurement ledger entry* records:

(i) the measured quantity;

(ii) the ideal repaired quantity being approximated;

(iii) the measurement procedure;

(iv) the error bound  $\delta^{\text{meas}}$ ;

(v) the window, threshold, and monitored degree;

(vi) how the error enters the  $\delta$ -ledger;

(vii) artifact references supporting the measurement.

**Remark 3.25** (Measurement error belongs to the ledger). If a certificate relies on measured Betti integrals or measured Betti curves, the measurement error must enter the  $\delta$ -ledger as a declared component:

$$\delta^{\text{meas}} \subset \Sigma\delta.$$

## J.6. Measurement-level stability

**Definition 3.26** (Measurement-level stability). A measurement-level quantity  $Q$  is *stable under perturbation* with modulus  $\omega$  if, whenever the underlying repaired persistence objects  $M, N$  satisfy

$$d_{\text{int}}(M, N) \leq \varepsilon,$$

one has

$$|Q(M) - Q(N)| \leq \omega(\varepsilon).$$

**Proposition 3.27** (Betti integral stability under bounded barcode perturbation). Assume two repaired persistence objects  $M, N$  on a bounded window  $W$  have barcodes matched within interleaving distance  $\varepsilon$ , and assume the number of monitored bars crossing  $W$  is bounded by  $R$ . Then the corresponding Betti integrals satisfy an estimate of the form

$$|\mathcal{B}_i(M; W) - \mathcal{B}_i(N; W)| \leq C_W R \varepsilon,$$

where  $C_W$  is a window-dependent constant determined by the chosen matching convention.

*Proof sketch.* Under an  $\varepsilon$ -matching of barcodes, each matched interval endpoint shifts by at most  $\varepsilon$ , so the contribution of each matched bar to the integral changes by at most a constant multiple of  $\varepsilon$ . If at most  $R$  monitored bars interact with  $W$ , the total change is bounded by  $C_W R \varepsilon$ . Unmatched bars must be  $\varepsilon$ -small and are controlled by the same barcode stability estimate.  $\square$

**Remark 3.28** (Bounded-bar assumption). The bounded-bar assumption is essential. Without a bound on the number or total mass of bars interacting with  $W$ , a small interleaving perturbation can accumulate over many bars and produce a large integral discrepancy.

**Definition 3.29** (Measurement stability certificate). A *measurement stability certificate* for a Betti integral consists of:

- (i) a perturbation or approximation bound  $\varepsilon$ ;
- (ii) a modulus  $\omega$ ;
- (iii) a resulting measurement error bound

$$\delta_{\mathcal{B}}^{\text{meas}} := \omega(\varepsilon);$$

- (iv) the bounded-bar or bounded-mass assumptions used;
- (v) artifact references supporting the estimate.

**Remark 3.30** (Measurement stability is not Core stability). Measurement-level stability of Betti integrals is not the same as Core stability of tower diagnostics. A Betti integral may be stable while Type IV diagnostics fail, or unstable while Core diagnostics still pass.

## J.7. Betti integral and discretization

**Definition 3.31** (Discretized Betti measurement). A *discretized Betti measurement* is a Betti curve or Betti integral computed from discretized data  $X_h$ , as in Appendix I. It must be associated with a realized discretized object

$$\mathfrak{R}_h(X_h)$$

and repaired evaluation records

$$\text{Eval}_{i,W,\tau}(\mathfrak{R}_h(X_h)).$$

**Definition 3.32** (Combined discretization-measurement error). The combined error for a discretized measured Betti integral is

$$\delta_{\text{total}}^{\text{BI}} := \delta^{\text{disc}} \oplus \delta_{\mathcal{B}}^{\text{meas}},$$

where  $\oplus$  is the declared ledger aggregation law.

**Proposition 3.33** (Discretized measured Betti evidence must use combined error). *If a certificate record relies on a discretized measured Betti integral, then both discretization error and measurement error must be included in the ledger.*

*Proof.* Discretization error measures the discrepancy between the discretized object and the continuum or intended repaired Core object. Measurement error measures the discrepancy between the measured aggregate and the ideal aggregate on that object. Both affect the reliability of the Betti-integral evidence. Therefore both must be ledgered.  $\square$

**Remark 3.34** (Relation to Appendix I). Appendix I handles the grid-to-continuum component. This appendix handles the measurement-level aggregation component. When both are present, the certificate record must include both.

## J.8. Betti integral and budget admissibility

**Definition 3.35** (Betti-integral adjusted budget). If Betti-integral evidence is used in a certificate record, the adjusted budget is

$$\Sigma\delta_{\text{BI}} := \Sigma\delta_{\text{base}} \oplus \delta_{\mathcal{B}}^{\text{meas}},$$

or, in the discretized measured case,

$$\Sigma\delta_{\text{BI}} := \Sigma\delta_{\text{base}} \oplus \delta^{\text{disc}} \oplus \delta_{\mathcal{B}}^{\text{meas}}.$$

**Proposition 3.36** (Betti-integral evidence must pass the adjusted budget). *A certificate record relying on Betti-integral evidence may pass Audit Mode only if*

$$\Sigma\delta_{\text{BI}} < \text{gap}_{\tau}.$$

*Proof.* By the Core audit discipline, all measurement and approximation errors used in the certificate must be included in the total budget. Thus the relevant adjusted budget must satisfy the strict inequality

$$\Sigma\delta < \text{gap}_{\tau}.$$

□

**Remark 3.37** (Betti evidence is not budget-free). A Betti integral is often a compressed measurement. Compression does not eliminate error. Any measurement-level use must be budgeted.

## J.9. Betti integral and monitored Type IV

**Definition 3.38** (Betti Type IV proxy). A *Betti Type IV proxy* is a Betti-curve or Betti-integral quantity intended to correlate with possible monitored generic Type IV obstruction.

**Proposition 3.39** (Betti Type IV proxy is not Type IV diagnosis). *A Betti Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0).$$

*Proof.* Monitored Type IV diagnostics are defined by generic fiber dimensions of kernel and cokernel defect profiles of declared repaired tower comparison artifacts

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty}).$$

A Betti integral summarizes local repaired persistence mass. It does not, by itself, compute the kernel or cokernel defect profiles of the tower comparison artifact. Therefore it is not the Type IV diagnostic. □

**Remark 3.40** (Correct use of Betti Type IV proxies). A Betti Type IV proxy may suggest that tower diagnostics should be computed or rechecked. It may not replace  $\mu$ ,  $u$  or the required tower comparison artifact.

## J.10. Betti integral and stable bands

**Definition 3.41** (Betti-stable band). A threshold interval

$$B \subset (0, \infty)$$

is called a *Betti-stable band* if the Betti integral

$$\mathcal{B}_{i,W,\tau}(F)$$

varies within a declared measurement tolerance for all  $\tau \in B$ .

**Proposition 3.42** (Betti-stable band is not Core stable band). *A Betti-stable band is not a Core stable band unless the monitored generic Type IV diagnostics satisfy*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

*throughout the band, relative to declared tower comparison artifacts.*

*Proof.* Betti stability concerns stability of a measurement-level aggregate. Core stability concerns vanishing of monitored generic tower diagnostics. These are distinct notions. Therefore Betti stability alone does not imply Core stability.  $\square$

**Remark 3.43** (Use of Betti-stable bands). Betti-stable bands may be useful for proposing candidate stable bands. They must still be verified by the Core diagnostic machinery.

## J.11. Betti integral artifact requirements

**Definition 3.44** (Betti measurement artifact). A *Betti measurement artifact* is an artifact supporting a Betti curve or Betti integral. Examples include:

- (i) repaired evaluation records;
- (ii) barcode data;
- (iii) sampled Betti curves;
- (iv) integration grids;
- (v) numerical quadrature records;
- (vi) measurement-error estimates;
- (vii) discretization-error estimates;
- (viii) plots used for advisory interpretation;
- (ix) source data used to compute the measurement.

**Definition 3.45** (Betti measurement manifest entry). A *Betti measurement manifest entry* records:

- (i) the monitored degree  $i$ ;
- (ii) the window  $W$ ;
- (iii) the threshold  $\tau$ ;

(iv) the repaired profile

$$\text{Eval}_{i,w,\tau}(F)$$

from which the measurement is derived;

(v) the Betti curve or Betti integral being measured;

(vi) the measurement method;

(vii) the measurement error bound;

(viii) the ledger entry into which the error is placed;

(ix) artifact references.

**Proposition 3.46** (Betti measurement artifacts are required for audit readability). *A certificate record relying on Betti-integral evidence is not audit-readable unless it includes Betti measurement artifact references.*

*Proof.* Without artifact references, an auditor cannot reconstruct the repaired evaluation profile, the measured Betti curve, the integration procedure, the error bound, or the ledger contribution. Thus the record fails the audit-readable requirements of Appendix G.  $\square$

## J.12. Summary of Betti-integral measurement layer

**Theorem 3.47** (Appendix J Betti-measurement package). *Within the Technical Extension layer, the following package is available.*

(i) *Windowed repaired Betti curves are computed from*

$$\text{Eval}_{i,w,\tau}(F) = T_{\tau}^{\text{bar}}(\mathbb{W}_w \mathbf{P}_i(F)).$$

(ii) *The Betti integral summarizes total repaired Betti mass over a window.*

(iii) *Betti integrals are measurement-level invariants, not Core gates.*

(iv) *Under constructibility and endpoint hypotheses, zero degree-1 Betti integral may witness*

$$\text{Eval}_{1,w,\tau}(F) = 0.$$

(v) *For an admissible representative  $C_{\tau,w}F$ , this also gives*

$$\text{PH}_1(C_{\tau,w}F) = 0.$$

(vi) *Such a witness does not replace the full B-Gate<sup>+</sup>.*

(vii) *Measurement error must be entered into the ledger as*

$$\delta^{\text{meas}}.$$

(viii) *Discretized measured Betti evidence must include both discretization and measurement error.*

(ix) *Any Betti-integral evidence used in an audited certificate must pass the adjusted budget condition*

$$\Sigma \delta_{\text{BI}} < \text{gap}_{\tau}.$$

(x) *Betti Type IV proxies do not replace*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}})$$

*or the tower comparison artifact*

$$\phi_{i,W,\tau}.$$

(xi) *Betti-stable bands do not become Core stable bands without monitored diagnostic verification.*

(xii) *Betti measurement artifacts are required for audit readability.*

*Proof.* Items (i)–(iii) follow from the definitions of repaired Betti curves and Betti integrals. Item (iv) is Proposition 3.17. Item (v) follows from admissible representative equivalence. Item (vi) follows from Remark 3.18. Items (vii)–(ix) follow from the measurement ledger and adjusted budget propositions. Item (x) is Proposition 3.39. Item (xi) is Proposition 3.42. Item (xii) is Proposition 3.46.  $\square$

### J.13. Explicit exclusions

The following are not asserted in Appendix J.

- No Betti integral is treated as a Core gate.
- No Betti curve replaces B-Gate<sup>+</sup>.
- No Betti integral replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No Betti integral replaces the eligible Ext<sup>1</sup>-clause.
- No Betti integral replaces monitored Type IV diagnostics.
- No Betti integral replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No Betti integral replaces the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

- No positive or negative Betti measurement is interpreted without ledgered measurement error.
- No raw Betti curve is treated as a repaired Core object.
- No Betti-stable band is treated as a Core stable band without diagnostic verification.
- No Betti Type IV proxy is treated as the actual computation of

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No discretized Betti measurement is accepted without discretization-error accounting.
- No plot or visual profile is treated as a proof.

- No measurement-level invariant is promoted to theorem status by itself.
- No global exactness of  $T_\tau^{\text{bar}}$  is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift  $C_\tau$  is assumed.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved here.

## J.14. Provenance and status

The material in this appendix depends on:

- (i) the repaired window-evaluation order of Chapters 2 and 5;
- (ii) the admissible representative discipline of Appendix B;
- (iii) the eligible one-way bridge of Appendix C;
- (iv) the audit and ledger discipline of Chapter 6;
- (v) the monitored Type IV diagnostic calculus of Appendix D;
- (vi) the advisory-invariant discipline of Appendix H;
- (vii) the discretization-stability discipline of Appendix I;
- (viii) the manifest requirements of Appendix G.

Its role is to allow Betti-integral and measurement-level evidence to support Core certification only through explicit measurement accounting and audit re-entry.

**Status labels for Appendix J.** *Technical extension definitions:* Definitions [3.4](#), [3.5](#), [3.7](#), [3.9](#), [3.10](#), [3.11](#), [3.16](#), [3.21](#), [3.22](#), [3.23](#), [3.24](#), [3.26](#), [3.29](#), [3.31](#), [3.32](#), [3.35](#), [3.38](#), [3.41](#), [3.44](#), [3.45](#).

*Technical extension propositions/theorems:* Propositions [3.14](#), [3.17](#), [3.19](#), [3.27](#), [3.33](#), [3.36](#), [3.39](#), [3.42](#), [3.46](#), Theorem [3.47](#).

*Operational cautions:* Remarks [3.1](#), [3.2](#), [3.34](#), [3.6](#), [3.8](#), [3.12](#), [3.13](#), [3.15](#), [3.18](#), [3.20](#), [3.25](#), [3.28](#), [3.30](#), [3.34](#), [3.40](#), [3.43](#).

*Explicit exclusions:* Section J.13.

## 4 Appendix K: AWFS and Controlled Commutation

### K.1. Scope and non-Core status

This appendix records optional controlled-commutation structures that may be used to compare different operational orders in an extended AK pipeline. The main examples are:

|                     |                                   |                                      |
|---------------------|-----------------------------------|--------------------------------------|
| window restriction, | barcode-level threshold deletion, | admissible representative selection, |
|                     | realization,                      | normalization, transfer.             |



The material here belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Controlled commutation is not Core equality unless its defect is zero in the declared Core sense.

In particular, an approximately commuting square, soft-commuting diagram, AWFS factorization, or controlled 2-cell does not replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_\tau.$$

All Core-relevant persistence data remain governed by the repaired windowed evaluation:

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

**Remark 4.1** (Technical extension status). The controlled-commutation structures introduced in this appendix are optional organizational tools. They may support implementation, diagrammatic accounting, or controlled comparison between alternative pipelines. They are not part of the repaired Core verifier.

**Remark 4.2** (Why this appendix exists). Chapter 2 intentionally excluded global exact collapse functors, unconditional crop–collapse commutation, AWFS structures, soft-commuting protocols, and product-ledger refinements from the repaired Core. This appendix reintroduces controlled commutation only as technical extension material, with strict non-upgrade rules.

**Remark 4.3** (No restoration of the old exact-collapse Core). Nothing in this appendix restores the older interpretation of positive-threshold collapse as a global exact reflector or Serre localization. Whenever a diagram uses threshold deletion, it must use the repaired barcode-level operation

$$T_\tau^{\text{bar}}$$

unless a separately declared exact-admissible regime is explicitly supplied.

## K.2. Pipeline diagrams and commutation defects

**Definition 4.4** (Pipeline diagram). A *pipeline diagram* is a diagram in a declared category, bicategory, or typed operational environment whose vertices are objects appearing in an AK pipeline and whose arrows represent operations such as:

$$W_W, \quad T_\tau^{\text{bar}}, \quad C_{\tau,W}, \quad \mathcal{R}, \quad \mathfrak{N}, \quad \mathfrak{T}.$$

Here  $\mathfrak{N}$  may denote a normalization operation and  $\mathfrak{T}$  a transfer operation when such extensions are declared.

**Definition 4.5** (Strictly commuting square). A square

$$\begin{array}{ccc} A & \xrightarrow{f} & B \\ g \downarrow & & \downarrow g' \\ C & \xrightarrow{f'} & D \end{array}$$

is *strictly commuting* if

$$g' \circ f = f' \circ g$$

in the declared category or if a declared canonical isomorphism identifies the two composites in the declared equivalence regime.

**Definition 4.6** (Commutation defect). A *commutation defect* for a square is a declared measure

$$\Delta_{\square}$$

of the discrepancy between the two composites:

$$g' \circ f \quad \text{and} \quad f' \circ g.$$

The defect may be measured as an interleaving distance, a norm, a homotopy defect, a 2-cell cost, a ledger value, a representative discrepancy, a comparison-artifact discrepancy, or another declared quantity.

**Definition 4.7** (Zero commutation defect). A commutation defect is *zero* if

$$\Delta_{\square} = 0$$

in the declared defect semantics. Only then may the square be treated as strictly commuting for the corresponding declared purpose.

**Remark 4.8** (Defect semantics must be declared). The symbol  $\Delta_{\square}$  has no intrinsic meaning until its target, measurement rule, and interpretation are declared. A numerical defect, categorical 2-cell, homotopy witness, interleaving bound, and ledger contribution are different kinds of data. They must not be conflated.

### K.3. Soft commutation

**Definition 4.9** (Soft-commuting square). A square is *soft-commuting* if its commutation defect satisfies a declared bound

$$\Delta_{\square} \leq \delta_{\square}.$$

The quantity  $\delta_{\square}$  is the declared commutation budget for the square.

**Definition 4.10** (Soft-commuting diagram). A diagram is *soft-commuting* if every declared comparison square in the diagram soft-commutes with its own declared commutation budget.

**Remark 4.11** (Soft commutation is not equality). Soft commutation does not mean equality. It means equality up to declared defect. If the defect is used in a certificate record, it must be entered into the relevant ledger.

**Definition 4.12** (Commutation ledger component). A *commutation ledger component* is a budget entry

$$\delta^{\text{comm}}$$

recording the total commutation defect contributed by one or more soft-commuting diagrams.

**Proposition 4.13** (Soft commutation affects Core only through the ledger). A *soft-commuting diagram affects Core audit only if its commutation defect is explicitly entered into the  $\delta$ -ledger.*

*Proof.* Core audit depends on repaired evaluation records, gate verdicts, monitored diagnostics, required artifacts, and the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

A soft-commutation defect is not itself a repaired evaluation record, gate verdict, or monitored Type IV diagnostic. It affects the Core only if declared as a budget component contributing to  $\Sigma\delta$ .  $\square$

#### K.4. Controlled commutation

**Definition 4.14** (Controlled commutation). A square or diagram has *controlled commutation* if:

- (i) the relevant composites are defined;
- (ii) the commutation defect  $\Delta_\square$  is declared;
- (iii) a bound

$$\Delta_\square \leq \delta_\square$$

is proved, assumed, or artifact-supported;

- (iv) the defect is either shown to vanish or entered into the appropriate ledger component.

**Definition 4.15** (Controlled commutation certificate). A *controlled commutation certificate* is a record containing:

- (i) the diagram being compared;
- (ii) the two composites being compared;
- (iii) the defect semantics;
- (iv) the defect bound;
- (v) the ledger treatment of the defect;
- (vi) artifact references supporting the comparison.

**Proposition 4.16** (Controlled commutation is audit-readable when ledged). A *controlled commutation certificate* is audit-readable if its defect semantics, bound, ledger treatment, and artifact references are declared.

*Proof.* Audit readability requires that the verifier can reconstruct the claimed comparison and determine how its defect affects the certificate. The declared defect semantics, bound, ledger entry, and artifacts supply this information.  $\square$

**Remark 4.17** (Controlled does not mean harmless). A controlled defect may still consume the entire safety margin. Thus controlled commutation is compatible with Core audit only if the total budget remains below the declared gap:

$$\Sigma\delta < \text{gap}_\tau.$$

#### K.5. Algebraic weak factorization systems

**Definition 4.18** (Weak factorization system). A *weak factorization system* on a category  $C$  consists of two classes of morphisms

$$(\mathcal{L}, \mathcal{R})$$

such that:

- (i) every morphism factors as

$$f = r \circ l$$

with  $l \in \mathcal{L}$  and  $r \in \mathcal{R}$ ;

- (ii)  $\mathcal{L}$  consists of morphisms having the left lifting property with respect to  $\mathcal{R}$ ;

(iii)  $\mathcal{R}$  consists of morphisms having the right lifting property with respect to  $\mathcal{L}$ .

**Definition 4.19** (Algebraic weak factorization system). An *algebraic weak factorization system*  $A$  on a category  $C$  is a weak factorization system equipped with functorial factorization and algebraic structure on the left and right factors. Informally, it promotes factorization from mere existence to structured, functorial data.

**Definition 4.20** (AWFS-controlled pipeline). An *AWFS-controlled pipeline* is a pipeline diagram in which selected morphisms are factorized using a declared AWFS

$$A = (L, R)$$

and the resulting factorization data are used to organize controlled lifting, replacement, comparison, or representative selection.

**Remark 4.21** (AWFS is optional). The Core does not require an AWFS. AWFS data may improve implementation discipline or categorical organization, but they do not define repaired collapse, B-Gate<sup>+</sup>, Type IV diagnostics, or audit pass/reject.

## K.6. AWFS and admissible representatives

**Definition 4.22** (AWFS-assisted representative). An *AWFS-assisted representative* is an admissible representative

$$C_{\tau, WF}$$

whose construction or comparison is organized by a declared AWFS factorization.

**Proposition 4.23** (AWFS-assisted representative does not improve Core status). *An AWFS-assisted representative has the same Core status as any other admissible representative: it is valid only up to the representative compatibility required in Appendix B.*

*Proof.* The Core uses admissible representatives through repaired profile compatibility:

$$\mathbf{P}_i(C_{\tau, WF}) \cong \text{Eval}_{i, W, \tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

An AWFS may organize the construction of a representative, but it does not change the required compatibility condition. Therefore it does not upgrade the Core status of the representative.  $\square$

**Remark 4.24** (AWFS is construction support). AWFS-assisted representative construction may be valuable. However, its value is organizational or technical. The Core still checks only repaired evaluation compatibility, representative compatibility, diagnostics, required artifacts, and budget conditions.

## K.7. Controlled commutation with repaired collapse

**Definition 4.25** (Repaired collapse-commutation square). A *repaired collapse-commutation square* is a square comparing an operation  $Q$  with barcode-level threshold deletion:

$$\begin{array}{ccc} M & \xrightarrow{Q} & Q(M) \\ T_{\tau}^{\text{bar}} \downarrow & & \downarrow T_{\tau}^{\text{bar}} \\ T_{\tau}^{\text{bar}} M & \xrightarrow{Q_{\tau}} & Q_{\tau}(T_{\tau}^{\text{bar}} M). \end{array}$$

The square is interpreted at the barcode-level or in a declared profile regime, not as a global exact functorial square unless exact-admissibility is separately declared.

**Definition 4.26** (Collapse-commutation defect). The *collapse-commutation defect* of a repaired collapse-commutation square is a declared quantity

$$\Delta_{T,Q}$$

measuring the discrepancy between:

$$T_\tau^{\text{bar}} Q(M) \quad \text{and} \quad Q_\tau T_\tau^{\text{bar}}(M).$$

**Proposition 4.27** (Collapse-commutation defect must be ledgered). *If a certificate record relies on replacing*

$$T_\tau^{\text{bar}} Q(M)$$

by

$$Q_\tau T_\tau^{\text{bar}}(M)$$

*under nonzero controlled defect, then the defect must be entered into the  $\delta$ -ledger.*

*Proof.* The replacement changes the object used for repaired Core-relevant evaluation by a declared defect. If the defect is nonzero, it contributes to the discrepancy between the ideal repaired pipeline and the implemented pipeline. By Chapter 6, such discrepancies must be included in the ledger.  $\square$

**Remark 4.28** (Strict commutation as a special case). If

$$\Delta_{T,Q} = 0$$

in the declared defect semantics, then the square may be treated as commuting for the declared purpose. If not, the defect must be budgeted.

**Remark 4.29** (No exact-collapse restoration). A repaired collapse-commutation square does not imply that  $T_\tau^{\text{bar}}$  is exact, functorial on all morphisms, or a Serre localization. It only records a controlled comparison in the declared profile context.

## K.8. Controlled commutation with window restriction

**Definition 4.30** (Window-commutation square). A *window-commutation square* compares an operation  $Q$  with window restriction:

$$\begin{array}{ccc} M & \xrightarrow{Q} & Q(M) \\ W_W \downarrow & & \downarrow W_W \\ W_W M & \xrightarrow{Q_W} & Q_W(W_W M). \end{array}$$

**Definition 4.31** (Window-commutation defect). The *window-commutation defect* is a declared quantity

$$\Delta_{W,Q}$$

measuring the discrepancy between applying  $Q$  before window restriction and applying the corresponding windowed operation  $Q_W$  after restriction.

**Definition 4.32** (Crop-collapse commutation defect). The *crop-collapse commutation defect* is the special case measuring the discrepancy between

$$W_W T_\tau^{\text{bar}}(M) \quad \text{and} \quad T_\tau^{\text{bar}}(W_W M).$$

**Remark 4.33** (Relation to crop–collapse noncommutation). Appendix A established that, in general,

$$W_W T_\tau^{\text{bar}} \not\equiv T_\tau^{\text{bar}} W_W.$$

Thus crop–collapse commutation cannot be assumed. If it is used approximately, it must be treated as a controlled commutation claim with a declared defect.

**Proposition 4.34** (Window-commutation defect is not endpoint bookkeeping). *A nonzero window-commutation defect cannot be dismissed as endpoint bookkeeping unless the defect is explicitly shown to vanish under the declared endpoint convention.*

*Proof.* Window restriction can shorten bars and thereby alter the effect of length-threshold deletion or other operations. This is a genuine persistence-level change, not merely notation. Therefore a nonzero defect must either be proved absent or budgeted.  $\square$

**Remark 4.35** (Default evaluation avoids hidden commutation). The repaired Core avoids hidden crop–collapse commutation by defining

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F))$$

as the fixed evaluation order. Any alternative order must be justified by a controlled commutation certificate or by an exact-admissible condition.

## K.9. Controlled commutation with realization

**Definition 4.36** (Realization-commutation square). A *realization-commutation square* compares an operation  $Q$  on filtered objects with an operation  $\bar{Q}$  on realized derived objects:

$$\begin{array}{ccc} F & \xrightarrow{Q} & Q(F) \\ \mathcal{R} \downarrow & & \downarrow \mathcal{R} \\ \mathcal{R}(F) & \xrightarrow{\bar{Q}} & \bar{Q}(\mathcal{R}(F)). \end{array}$$

**Definition 4.37** (Realization-commutation defect). The *realization-commutation defect* is a declared quantity

$$\Delta_{\mathcal{R},Q}$$

measuring the discrepancy between:

$$\mathcal{R}(Q(F)) \quad \text{and} \quad \bar{Q}(\mathcal{R}(F)).$$

**Proposition 4.38** (Realization commutation does not replace eligibility). *A realization-commutation square, even if controlled, does not replace the eligibility requirements of Chapter 3 and Appendix C.*

*Proof.* Eligibility requires that

$$\mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod})$$

with the edge-realization condition linking  $H^{-1}$  to the repaired residual degree-1 persistence edge. A controlled commutation square involving  $\mathcal{R}$  does not by itself establish amplitude  $[-1, 0]$  or edge realization. Therefore it does not replace eligibility.  $\square$

**Remark 4.39** (Eligibility remains mandatory). Whenever the  $\text{Ext}^1$ -clause is invoked, eligibility must be checked directly. Controlled realization diagrams may support the construction, but they do not license the  $\text{Ext}^1$ -edge by themselves.

## K.10. Two-cell controlled commutation

**Definition 4.40** (Controlled 2-cell). In a bicategorical or 2-categorical setting, a *controlled 2-cell* is a 2-morphism

$$\alpha : g' \circ f \Rightarrow f' \circ g$$

together with a declared cost or defect

$$\delta(\alpha).$$

**Definition 4.41** (2-cell ledger). A *2-cell ledger* records the defects of controlled 2-cells:

$$\delta(\alpha_1), \dots, \delta(\alpha_m),$$

and aggregates them into a total 2-cell commutation budget

$$\delta^{2\text{-cell}}.$$

**Remark 4.42** (Relation to Appendix L). Appendix L may specialize this idea to Mirror/Transfer 2-cell ledgers. The present appendix gives the general controlled-commutation framework.

**Proposition 4.43** (2-cell control affects audit only through ledger). *A controlled 2-cell affects Core audit only through its declared ledger contribution.*

*Proof.* A 2-cell cost is not a Core gate verdict and not a monitored Type IV diagnostic. If it affects the correctness of a pipeline comparison, it must be entered into the budget ledger. Only then can it affect the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

□

## K.11. Composition of commutation defects

**Definition 4.44** (Sequential defect aggregation). Given a sequence of controlled commutation steps with defects

$$\delta_1, \dots, \delta_m,$$

their sequential aggregate is

$$\delta^{\text{seq}} := \delta_1 \oplus \dots \oplus \delta_m$$

in the declared budget quantale.

**Definition 4.45** (Parallel defect aggregation). Given parallel controlled commutation comparisons with defects

$$\delta_\alpha$$

indexed by a finite set  $A$ , their parallel aggregate is

$$\delta^{\text{par}} := \bigoplus_{\alpha \in A} \delta_\alpha$$

or another declared aggregation law appropriate to the chosen quantale.

**Proposition 4.46** (Defect aggregation must match ledger semantics). *Sequential or parallel aggregation of commutation defects is admissible only when it uses the declared ledger aggregation law.*

*Proof.* The  $\delta$ -ledger determines how budget components combine. If commutation defects are included in the ledger, they must be aggregated using that declared law. Otherwise the resulting total budget is not audit-readable.  $\square$

**Remark 4.47** (No hidden additive convention). This appendix does not assume ordinary addition unless ordinary addition is the declared quantale operation. Different ledger semantics may use maximum, weighted sum, lexicographic order, or other aggregation laws.

## K.12. Controlled commutation and certificate records

**Definition 4.48** (Commutation manifest entry). A *commutation manifest entry* records:

- (i) the diagram being compared;
- (ii) the two composites;
- (iii) the defect semantics;
- (iv) the defect value or bound;
- (v) the ledger component receiving the defect;
- (vi) artifact references supporting the comparison;
- (vii) whether the defect is zero, budgeted, or non-admissible.

**Definition 4.49** (Commutation status). A commutation manifest entry has one of the following statuses:

- (i) *strict*: the defect is zero;
- (ii) *controlled*: the defect is nonzero but budgeted;
- (iii) *uncontrolled*: no admissible defect bound is declared;
- (iv) *failed*: the defect bound exceeds the available budget margin.

**Proposition 4.50** (Uncontrolled commutation cannot support Core certification). *An uncontrolled commutation claim cannot be used to support a Core certificate record.*

*Proof.* If no admissible defect bound is declared, then the discrepancy between the two composites is not audit-readable. A non-audit-readable comparison cannot be used in a certified proof object.  $\square$

**Proposition 4.51** (Failed commutation defeats affected pipeline use). *If a commutation defect exceeds the available budget margin, then any certificate record depending on that commutation replacement fails Audit Mode unless an alternative verified path is provided.*

*Proof.* A certificate depending on the replacement must include the commutation defect in  $\Sigma\delta$ . If the defect causes

$$\Sigma\delta \geq \text{gap}_\tau,$$

then the budget condition fails. Thus the affected certificate cannot pass unless it avoids that replacement through another verified route.  $\square$



### K.13. Relation to advisory invariants

**Definition 4.52** (Advisory commutation indicator). An *advisory commutation indicator* is a heuristic, spectral, numerical, diagrammatic, or measurement-level quantity suggesting that a diagram may approximately commute.

**Proposition 4.53** (Advisory commutation indicator is not commutation proof). *An advisory commutation indicator does not establish controlled commutation.*

*Proof.* Controlled commutation requires declared defect semantics, a defect bound, and ledger treatment. An advisory indicator may suggest that such a bound exists, but it does not provide the bound itself.  $\square$

**Remark 4.54** (Correct use of advisory indicators). An advisory commutation indicator may guide where to build a controlled commutation certificate. It may not replace the certificate.

### K.14. Summary of AWFS and controlled commutation

**Theorem 4.55** (Appendix K controlled-commutation package). *Within the Technical Extension layer, the following package is available.*

- (i) *Pipeline diagrams may be compared by strict or controlled commutation.*
- (ii) *A commutation defect must have declared semantics.*
- (iii) *Soft commutation means bounded defect, not equality.*
- (iv) *Nonzero commutation defects affect Core audit only through the  $\delta$ -ledger.*
- (v) *AWFS structures may organize representative, lift, or factorization data, but they are optional and non-Core.*
- (vi) *AWFS-assisted representatives have the same Core status as ordinary admissible representatives.*
- (vii) *Collapse-commutation, window-commutation, crop-collapse commutation, and realization-commutation defects must be budgeted if used in certification.*
- (viii) *Controlled 2-cells affect audit only through declared 2-cell ledger contributions.*
- (ix) *Defect aggregation must follow the declared ledger semantics.*
- (x) *Uncontrolled commutation cannot support Core certification.*
- (xi) *Advisory commutation indicators do not establish controlled commutation.*
- (xii) *No controlled commutation statement replaces the repaired evaluation*

$$\text{Eval}_{i,w,\tau}(F).$$

*Proof.* Items (i)–(iii) follow from the definitions of pipeline diagrams, commutation defects, and soft commutation. Item (iv) is Proposition 4.13. Items (v)–(vi) follow from the AWFS definitions and Proposition 4.23. Item (vii) follows from the collapse, window, crop-collapse, and realization commutation defect propositions. Item (viii) is Proposition 4.43. Item (ix) is Proposition 4.46. Item (x) is Proposition 4.50. Item (xi) is Proposition 4.53. Item (xii) follows from the fixed repaired Core evaluation discipline.  $\square$

### K.15. Explicit exclusions

The following are not asserted in Appendix K.

- No AWFS is required for the Core.
- No AWFS structure replaces repaired barcode-level threshold deletion

$$T_{\tau}^{\text{bar}}.$$

- No AWFS-assisted representative has stronger Core status than an ordinary admissible representative.
- No soft-commuting diagram is treated as strictly commuting unless its defect is zero.
- No commutation defect is ignored when the corresponding replacement is used in certification.
- No uncontrolled commutation claim supports a Core certificate.
- No controlled commutation replaces B-Gate<sup>+</sup>.
- No controlled commutation replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No controlled commutation replaces monitored Type IV diagnostics.
- No controlled commutation replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No controlled commutation replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

- No realization-commutation square replaces eligibility.
- No crop-collapse commutation is assumed by default.
- No global exactness of  $T_{\tau}^{\text{bar}}$  is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift

$$C_{\tau} : \text{FiltCh}(( )k) \rightarrow \text{FiltCh}(( )k)$$

is assumed.

- No identity

$$C_{\tau,W}F = (C_{\tau}F)|_W$$

is assumed.

- No advisory commutation indicator is treated as a proof.
- No problem-specific arithmetic, PDE, geometric, or categorical conclusion is proved here.

## K.16. Provenance and status

The material in this appendix depends on:

- (i) the repaired barcode-level collapse and non-commutation cautions of Appendix A;
- (ii) the admissible representative discipline of Appendix B;
- (iii) the eligible realization regime of Appendix C;
- (iv) the monitored Type IV diagnostic calculus of Appendix D;
- (v) the audit and ledger discipline of Chapter 6;
- (vi) the advisory-invariant discipline of Appendix H;
- (vii) the manifest schema of Appendix G.

Its role is to allow controlled categorical and diagrammatic comparison without allowing those comparisons to contaminate Core verdicts.

**Status labels for Appendix K.** *Technical extension definitions:* Definitions [4.4](#), [4.5](#), [4.6](#), [4.7](#), [4.9](#), [4.10](#), [4.12](#), [4.14](#), [4.15](#), [4.18](#), [4.19](#), [4.20](#), [4.22](#), [4.25](#), [4.26](#), [4.30](#), [4.31](#), [4.32](#), [4.36](#), [4.37](#), [4.40](#), [4.41](#), [4.44](#), [4.45](#), [4.48](#), [4.49](#), [4.52](#).

*Technical extension propositions/theorems:* Propositions [4.13](#), [4.16](#), [4.23](#), [4.27](#), [4.34](#), [4.38](#), [4.43](#), [4.46](#), [4.50](#), [4.51](#), [4.53](#), Theorem [4.55](#).

*Operational cautions:* Remarks [4.1](#), [4.2](#), [4.3](#), [4.8](#), [4.11](#), [4.17](#), [4.21](#), [4.24](#), [4.28](#), [4.29](#), [4.33](#), [4.35](#), [4.39](#), [4.42](#), [4.47](#), [4.54](#).

*Explicit exclusions:* Section K.15.

## 5 Appendix L: Mirror/Transfer 2-cell Ledger

### L.1. Scope and non-Core status

This appendix records an optional ledger discipline for Mirror/Transfer comparisons. It specializes the controlled-commutation framework of Appendix K to diagrams in which one attempts to compare two pipelines through mirror-type or transfer-type operations.

The material here belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Mirror/Transfer compatibility is not a Core theorem unless its hypotheses, defects, and ledger effects are explicitly verified.

In particular, a Mirror/Transfer diagram, 2-cell, comparison functor, transfer square, or mirror correspondence does not replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_\tau.$$

All Core-relevant persistence data remain governed by the repaired windowed evaluation:

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

**Remark 5.1** (Technical extension status). Mirror/Transfer machinery may be useful for comparing different mathematical presentations, dual descriptions, categorical models, or computational pipelines. However, such machinery is not part of the repaired Core. It is a controlled extension that must respect Core sovereignty.

**Remark 5.2** (Relation to Appendix K). Appendix K introduced controlled commutation and 2-cell defect accounting in general. The present appendix specializes that framework to Mirror/Transfer situations, where the central data are comparison functors, transfer maps, mirror maps, and controlled 2-cells between composites.

**Remark 5.3** (No restoration of old exact-collapse language). This appendix does not restore the older exact-collapse notation or assumptions. In particular, Mirror/Transfer comparison with collapse is comparison with

$$T_{\tau}^{\text{bar}}$$

inside a declared barcode/profile regime, not with a global exact reflector unless a separate exact-admissible regime is explicitly declared.

## L.2. Mirror and Transfer settings

**Definition 5.4** (Mirror setting). A *Mirror setting* consists of two declared categories, bicategories, structured domains, or typed operational environments

$$\mathcal{A} \quad \text{and} \quad \mathcal{A}^{\vee},$$

together with a declared mirror correspondence

$$\text{Mir} : \mathcal{A} \dashrightarrow \mathcal{A}^{\vee}$$

which may be a functor, correspondence, span, adjunction-like datum, equivalence candidate, mirror map, or other explicitly declared comparison structure.

**Definition 5.5** (Transfer setting). A *Transfer setting* consists of two declared domains

$$\mathcal{A} \quad \text{and} \quad \mathcal{B},$$

together with a declared transfer operation

$$\text{Tr} : \mathcal{A} \dashrightarrow \mathcal{B}$$

intended to move objects, invariants, filtrations, profiles, representatives, diagnostics, or certificate-relevant data from  $\mathcal{A}$  to  $\mathcal{B}$ .

**Definition 5.6** (Mirror/Transfer pipeline). A *Mirror/Transfer pipeline* is a diagram involving one or more of:

$$W_W, \quad T_{\tau}^{\text{bar}}, \quad C_{\tau,W}, \quad \mathcal{R}, \quad \text{Mir}, \quad \text{Tr},$$

and possibly additional declared operations, where at least one path passes through a Mirror or Transfer operation.

**Remark 5.7** (Dashed arrows). The notation

$$\dashrightarrow$$

is intentional. A Mirror or Transfer operation need not be a strict functor on the entire ambient category. Its domain, target, variance, functoriality, and defect semantics must be declared before it may be used in a certificate record.

### L.3. Mirror/Transfer realization targets

**Definition 5.8** (Mirror/Transfer realization target). A Mirror/Transfer pipeline has a *realization target* if its output is declared to be one of:

- (i) a filtered object in  $\text{FiltCh}(\cdot)k$ ;
- (ii) a persistence object in  $\text{Pers}_k^{\text{cons}}$ ;
- (iii) a repaired windowed collapsed profile
$$\text{Eval}_{i,W,\tau}(F);$$
- (iv) an admissible representative
$$C_{\tau,W}F;$$
- (v) a realized derived object in  $D^b(k\text{-mod})$ ;
- (vi) a tower comparison artifact;
- (vii) a certificate-record component;
- (viii) an advisory or Search-side artifact.

**Definition 5.9** (Core-testable Mirror/Transfer output). A Mirror/Transfer output is *Core-testable* only if it is realized as data on which the Core can evaluate, or reconstruct, the repaired profile

$$\text{Eval}_{i,W,\tau}(-) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(-)),$$

or as a declared certificate-record component feeding into B-Gate<sup>+</sup>, monitored Type IV diagnostics, a tower comparison artifact, or the  $\delta$ -ledger.

**Remark 5.10** (No raw Mirror/Transfer substitution). A raw Mirror/Transfer output is not automatically a Core object. It must be typed into the Core pipeline through a declared realization target. This is the Mirror/Transfer analogue of the interface discipline of Chapter 8.

**Remark 5.11** (Core-testable does not mean Core-passing). Core-testable output merely has the right type for verifier-side use. It does not imply

$$\text{Eval}_{1,W,\tau} = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \Sigma\delta < \text{gap}_{\tau},$$

or any Core pass verdict.

### L.4. Mirror/Transfer comparison squares

**Definition 5.12** (Mirror comparison square). A *Mirror comparison square* is a diagram of the form

$$\begin{array}{ccc} X & \xrightarrow{\text{Mir}} & X^{\vee} \\ Q \downarrow & & \downarrow Q^{\vee} \\ QX & \xrightarrow{\text{Mir}_Q} & Q^{\vee} X^{\vee}, \end{array}$$

where  $Q$  and  $Q^{\vee}$  are declared operations on the two sides and  $\text{Mir}_Q$  is the induced or declared mirror comparison on the transformed objects.

**Definition 5.13** (Transfer comparison square). A *Transfer comparison square* is a diagram of the form

$$\begin{array}{ccc} X & \xrightarrow{\text{Tr}} & \text{Tr}(X) \\ Q \downarrow & & \downarrow \overline{Q} \\ QX & \xrightarrow{\text{Tr}_Q} & \overline{Q}\text{Tr}(X), \end{array}$$

where  $Q$ ,  $\overline{Q}$ , and  $\text{Tr}_Q$  are declared operations or induced transfer operations.

**Definition 5.14** (Mirror/Transfer commutation defect). A *Mirror/Transfer commutation defect* is a declared quantity

$$\Delta_{\text{MT}}$$

measuring the discrepancy between the two composites of a Mirror or Transfer comparison square.

**Remark 5.15** (Defect must be typed). The defect

$$\Delta_{\text{MT}}$$

must have a declared type: interleaving-distance value, norm bound, homotopy defect, 2-cell cost, categorical obstruction, representative discrepancy, comparison-artifact discrepancy, or ledger quantity. Untyped defect notation is not audit-readable.

## L.5. Mirror/Transfer 2-cells

**Definition 5.16** (Mirror 2-cell). A *Mirror 2-cell* is a 2-morphism

$$\alpha_{\text{Mir}} : Q^\vee \circ \text{Mir} \Rightarrow \text{Mir}_Q \circ Q$$

or a declared weak comparison between these composites.

**Definition 5.17** (Transfer 2-cell). A *Transfer 2-cell* is a 2-morphism

$$\alpha_{\text{Tr}} : \overline{Q} \circ \text{Tr} \Rightarrow \text{Tr}_Q \circ Q$$

or a declared weak comparison between these composites.

**Definition 5.18** (2-cell cost). A 2-cell cost is a declared ledger-valued quantity

$$\delta(\alpha)$$

attached to a Mirror or Transfer 2-cell  $\alpha$ , measuring the cost of using that 2-cell as a controlled comparison.

**Definition 5.19** (Zero-cost 2-cell). A Mirror/Transfer 2-cell is *zero-cost* if

$$\delta(\alpha) = 0$$

in the declared 2-cell cost semantics. Only zero-cost 2-cells may be treated as strict commutations for the declared purpose.

**Remark 5.20** (Weak comparison is not equality). A Mirror/Transfer 2-cell witnesses controlled comparison, not equality, unless its cost is zero and the declared semantics identify zero cost with strict equality for that purpose.

## L.6. The Mirror/Transfer 2-cell ledger

**Definition 5.21** (Mirror/Transfer 2-cell ledger). A *Mirror/Transfer 2-cell ledger* is a ledger component

$$\delta^{\text{MT}}$$

recording the aggregate cost of Mirror/Transfer comparison cells:

$$\delta^{\text{MT}} := \bigoplus_{\alpha \in A_{\text{MT}}} \delta(\alpha),$$

where  $A_{\text{MT}}$  is the declared finite or summable family of Mirror/Transfer 2-cells used in the certificate record.

**Definition 5.22** (Mirror/Transfer ledger entry). A *Mirror/Transfer ledger entry* records:

- (i) the Mirror/Transfer diagram;
- (ii) the 2-cell or weak comparison being used;
- (iii) the cost semantics;
- (iv) the cost value or bound;
- (v) the aggregation rule;
- (vi) how the cost enters the total  $\delta$ -ledger;
- (vii) artifact references supporting the comparison.

**Proposition 5.23** (Mirror/Transfer defects affect Core only through ledger). A *nonzero Mirror/Transfer 2-cell cost affects Core audit only if it is included in the declared  $\delta$ -ledger*.

*Proof.* Core audit is determined by repaired evaluation records, gate verdicts, monitored diagnostics, required artifacts, and the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

A Mirror/Transfer 2-cell cost is neither a repaired evaluation record nor a gate verdict nor a monitored Type IV diagnostic. It can affect Core audit only by contributing to  $\Sigma\delta$ .  $\square$

**Remark 5.24** (Ledger entry is mandatory if used). If a certificate relies on a nonzero Mirror/Transfer comparison, the corresponding cost must be recorded. Unrecorded Mirror/Transfer defects are not admissible in a Core certificate record.

## L.7. Mirror/Transfer and repaired collapse

**Definition 5.25** (Mirror-collapse comparison). A *Mirror-collapse comparison* compares the two paths:

$$X \mapsto \text{Mir}(X) \mapsto T_\tau^{\text{bar}} \text{Mir}(X)$$

and

$$X \mapsto T_\tau^{\text{bar}} X \mapsto \text{Mir}_\tau(T_\tau^{\text{bar}} X),$$

whenever these expressions are defined in the declared barcode/profile realization target.

**Definition 5.26** (Transfer-collapse comparison). A *Transfer-collapse comparison* compares the two paths:

$$X \mapsto \text{Tr}(X) \mapsto T_\tau^{\text{bar}} \text{Tr}(X)$$

and

$$X \mapsto T_\tau^{\text{bar}} X \mapsto \text{Tr}_\tau(T_\tau^{\text{bar}} X).$$

**Definition 5.27** (Collapse transfer defect). The *collapse transfer defect* is the Mirror/Transfer commutation defect associated to moving barcode-level threshold deletion across a Mirror or Transfer operation. It is denoted

$$\Delta_{T, \text{MT}}.$$

**Proposition 5.28** (Collapse transfer defect must be budgeted). *If a certificate record uses a Mirror/Transfer comparison to replace*

$$T_\tau^{\text{bar}} \text{MT}(X)$$

by

$$\text{MT}_\tau(T_\tau^{\text{bar}} X)$$

*under nonzero defect, then the defect must be included in the  $\delta$ -ledger.*

*Proof.* The replacement changes the repaired after-collapse data used by the pipeline. A nonzero discrepancy between the two routes is an admissible deviation only if it is declared and budgeted. Therefore it must enter  $\Sigma\delta$ .  $\square$

**Remark 5.29** (No automatic Mirror-collapse commutation). No automatic commutation between Mirror/Transfer operations and barcode-level threshold deletion is asserted. Such commutation must be either strict, zero-cost, or controlled with ledgered defect.

**Remark 5.30** (No exact-collapse claim). The notation in this section does not imply a global exact collapse functor. All statements are read at the barcode/profile level unless a separately declared exact-admissible regime is supplied.

## L.8. Mirror/Transfer and window restriction

**Definition 5.31** (Mirror-window comparison). A *Mirror-window comparison* compares applying a Mirror operation before and after window restriction:

$$W_W \text{Mir}(X) \quad \text{versus} \quad \text{Mir}_W(W_W X).$$

**Definition 5.32** (Transfer-window comparison). A *Transfer-window comparison* compares:

$$W_W \text{Tr}(X) \quad \text{versus} \quad \text{Tr}_W(W_W X).$$

**Definition 5.33** (Window transfer defect). The *window transfer defect* is the declared discrepancy

$$\Delta_{W, \text{MT}}$$

associated to moving window restriction across a Mirror or Transfer operation.

**Remark 5.34** (Window restriction is delicate). Window restriction can alter bar lengths and support data. Therefore Mirror/Transfer compatibility with windows is not automatic. It must be checked or budgeted.

**Proposition 5.35** (Window transfer defect must be declared). *Any certificate relying on a Mirror/Transfer window comparison must declare whether the comparison is strict, zero-cost, controlled, advisory, unavailable, or failed.*

*Proof.* The repaired windowed after-collapse object depends on the order of operations. If the certificate uses a route involving Mirror/Transfer and window restriction, an auditor must know whether the routes are equal, controlled, or not comparable. Thus the status must be declared.  $\square$



## L.9. Mirror/Transfer and repaired windowed evaluation

**Definition 5.36** (Mirror-evaluation comparison). A *Mirror-evaluation comparison* compares the two routes:

$$X \mapsto \text{Mir}(X) \mapsto \text{Eval}_{i,w,\tau}(\text{Mir}(X))$$

and

$$X \mapsto \text{Eval}_{i,w,\tau}(X) \mapsto \text{Mir}_{i,w,\tau}(\text{Eval}_{i,w,\tau}(X)),$$

whenever these expressions are defined in the declared profile regime.

**Definition 5.37** (Transfer-evaluation comparison). A *Transfer-evaluation comparison* compares the two routes:

$$X \mapsto \text{Tr}(X) \mapsto \text{Eval}_{i,w,\tau}(\text{Tr}(X))$$

and

$$X \mapsto \text{Eval}_{i,w,\tau}(X) \mapsto \text{Tr}_{i,w,\tau}(\text{Eval}_{i,w,\tau}(X)).$$

**Definition 5.38** (Evaluation transfer defect). The *evaluation transfer defect* is the declared discrepancy

$$\Delta_{\text{Eval},\text{MT}}$$

associated to moving the repaired windowed evaluation across a Mirror or Transfer operation.

**Proposition 5.39** (Evaluation transfer defect must be budgeted). *If a certificate uses a Mirror/Transfer comparison to replace one repaired evaluation route by another under nonzero defect, then*

$$\Delta_{\text{Eval},\text{MT}}$$

*must be included in the  $\delta$ -ledger.*

*Proof.* The repaired evaluation route determines the Core-relevant profile. Replacing one route by another under nonzero defect changes the evidence used by the certificate. By the audit discipline, this discrepancy must be ledgered.  $\square$

**Remark 5.40** (Preferred Core route). The preferred Core route is always to compute

$$\text{Eval}_{i,w,\tau}(-)$$

directly on the declared Core-readable output. Mirror/Transfer-based shortcuts require explicit comparison status and ledger treatment.

## L.10. Mirror/Transfer and realization

**Definition 5.41** (Mirror-realization comparison). A *Mirror-realization comparison* compares:

$$\mathcal{R}(\text{Mir}(F)) \quad \text{and} \quad \text{Mir}_D(\mathcal{R}(F)),$$

where  $\text{Mir}_D$  denotes a declared mirror operation at the realized derived level.

**Definition 5.42** (Transfer-realization comparison). A *Transfer-realization comparison* compares:

$$\mathcal{R}(\text{Tr}(F)) \quad \text{and} \quad \text{Tr}_D(\mathcal{R}(F)),$$

where  $\text{Tr}_D$  denotes a declared transfer operation at the realized derived level.

**Definition 5.43** (Realization transfer defect). The *realization transfer defect* is the declared discrepancy

$$\Delta_{\mathcal{R},\text{MT}}$$

associated to moving realization across a Mirror or Transfer operation.

**Proposition 5.44** (Mirror/Transfer realization does not replace eligibility). *A controlled Mirror/Transfer realization comparison does not replace the eligible realization requirements of Chapter 3 and Appendix C.*

*Proof.* Eligibility requires amplitude  $[-1, 0]$  and the edge-realization condition for the object

$$\mathcal{R}(C_{\tau,w}F).$$

A Mirror/Transfer comparison involving realization does not, by itself, establish these conditions. Therefore eligibility must still be separately verified whenever the  $\text{Ext}^1$ -clause is invoked.  $\square$

**Remark 5.45** (Eligibility remains local to the realized object). Even if a Mirror/Transfer pipeline is controlled, the realized object entering the  $\text{Ext}^1$ -edge must still be checked in the eligible regime.

## L.11. Mirror/Transfer and tower comparison artifacts

**Definition 5.46** (Mirror/Transfer tower comparison support). A Mirror/Transfer diagram provides *tower comparison support* if it helps construct, compare, or transport a declared tower comparison artifact

$$\phi_{i,w,\tau} : \text{ColimProfile}_{i,w,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,w,\tau}(F_\infty).$$

**Definition 5.47** (Mirror/Transfer tower artifact defect). The *Mirror/Transfer tower artifact defect* is a declared discrepancy

$$\Delta_{\phi,\text{MT}}$$

measuring the cost of transporting or comparing tower comparison artifacts through a Mirror/Transfer pipeline.

**Proposition 5.48** (Mirror/Transfer does not replace tower comparison artifacts). *A Mirror/Transfer diagram does not replace the required tower comparison artifact*

$$\phi_{i,w,\tau}.$$

*If Type IV diagnostics are invoked, the artifact must still be declared.*

*Proof.* Monitored Type IV diagnostics are computed from kernel and cokernel defect profiles of the declared comparison artifact

$$\phi_{i,w,\tau}.$$

A Mirror/Transfer diagram may help construct or compare such an artifact, but it is not itself the artifact unless explicitly declared as such and supplied with the required kernel/cokernel defect data.  $\square$

**Remark 5.49** (Artifact transport must be ledgered). If Mirror/Transfer machinery transports a tower comparison artifact only up to nonzero defect, that defect must be entered into the ledger before the transported artifact may support a certificate.

## L.12. Mirror/Transfer and certificate records

**Definition 5.50** (Mirror/Transfer certificate attachment). A *Mirror/Transfer certificate attachment* is an optional section of a certificate record documenting the Mirror/Transfer diagrams used to support or compare pipeline data.

**Definition 5.51** (Mirror/Transfer manifest entry). A *Mirror/Transfer manifest entry* contains:

- (i) the source and target domains;
- (ii) the Mirror or Transfer operation used;
- (iii) the realization target;
- (iv) the compared composites;
- (v) the controlled 2-cells;
- (vi) the 2-cell costs;
- (vii) the ledger aggregation rule;
- (viii) any evaluation transfer defect;
- (ix) any tower artifact transfer defect;
- (x) the final Mirror/Transfer comparison status;
- (xi) artifact references.

**Definition 5.52** (Mirror/Transfer comparison status). A Mirror/Transfer comparison has one of the following statuses:

- (i) *strict*: the comparison commutes on the declared nose;
- (ii) *zero-cost*: the comparison is weak but has zero declared cost;
- (iii) *controlled*: the comparison has nonzero but budgeted cost;
- (iv) *advisory*: the comparison is suggestive but not ledger-certified;
- (v) *unavailable*: no admissible comparison has been declared;
- (vi) *failed*: the cost exceeds the available budget margin.

**Proposition 5.53** (Only strict, zero-cost, or controlled comparisons may support certification). A *Mirror/Transfer comparison may support a Core certificate only if its status is strict, zero-cost, or controlled with ledged cost*.

*Proof.* A strict or zero-cost comparison introduces no defect for the declared purpose. A controlled comparison introduces a defect but records it in the ledger. Advisory, unavailable, and failed comparisons do not provide audit-readable support for a Core certificate.  $\square$

**Remark 5.54** (Advisory Mirror/Transfer is non-verdict). An advisory Mirror/Transfer comparison may guide exploration or suggest a candidate transfer. It does not support Core acceptance unless converted into a strict, zero-cost, or controlled ledger-certified comparison.

### L.13. Mirror/Transfer budget integration

**Definition 5.55** (Mirror/Transfer adjusted budget). If a certificate uses Mirror/Transfer comparison data, the adjusted budget is

$$\Sigma\delta_{\text{MT}} := \Sigma\delta_{\text{base}} \oplus \delta^{\text{MT}},$$

where  $\delta^{\text{MT}}$  is the aggregate Mirror/Transfer 2-cell ledger cost. If evaluation transfer or tower artifact transfer defects are separately recorded, they must also be included:

$$\Sigma\delta_{\text{MT}} := \Sigma\delta_{\text{base}} \oplus \delta^{\text{MT}} \oplus \Delta_{\text{Eval,MT}} \oplus \Delta_{\phi,\text{MT}},$$

with aggregation interpreted in the declared ledger semantics.

**Proposition 5.56** (Mirror/Transfer evidence must pass the adjusted budget). *A certificate record relying on nonzero Mirror/Transfer comparison cost may pass Audit Mode only if*

$$\Sigma\delta_{\text{MT}} < \text{gap}_{\tau}.$$

*Proof.* By the audit discipline of Chapter 6, all admissible deviations used by a certificate must be included in  $\Sigma\delta$ . If Mirror/Transfer comparison contributes nonzero defect, that defect is part of the total budget. Thus the adjusted budget must satisfy the strict pass condition.  $\square$

**Remark 5.57** (Mirror/Transfer is not budget-free). Mirror/Transfer comparisons often appear structurally elegant. However, elegance is not zero cost. Unless strict or zero-cost commutation is proved, the comparison consumes ledger budget.

### L.14. Mirror/Transfer and monitored Type IV

**Definition 5.58** (Mirror/Transfer Type IV proxy). A *Mirror/Transfer Type IV proxy* is a Mirror/Transfer-derived signal suggesting possible monitored generic tower defect.

**Definition 5.59** (Actual Type IV diagnostic in Mirror/Transfer context). The actual monitored generic Type IV diagnostic in a Mirror/Transfer context is still computed from a declared tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_{\bullet}) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_{\infty})$$

by taking the generic fiber dimensions of its kernel and cokernel defect profiles.

**Proposition 5.60** (Mirror/Transfer proxy is not Type IV diagnosis). *A Mirror/Transfer Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \text{ eq } (0, 0).$$

*Proof.* Monitored Type IV diagnostics are defined by the generic fiber dimensions of kernel and cokernel defect profiles of declared repaired tower comparison artifacts. A Mirror/Transfer proxy may correlate with such behavior but does not compute those defect profiles unless explicitly realized as the diagnostic calculation itself through the declared artifact

$$\phi_{i,W,\tau}.$$

$\square$

**Remark 5.61** (Correct use of Mirror/Transfer Type IV proxies). Mirror/Transfer proxies may suggest where to compute  $\mu$ ,  $u$ . They may not replace  $\mu$ ,  $u$ , nor the required tower comparison artifact.

## L.15. Mirror/Transfer as interface material

**Definition 5.62** (Problem-specific Mirror/Transfer implication). A *problem-specific Mirror/Transfer implication* is a statement of the form

$$\text{Mirror/Transfer compatibility} \implies \text{problem-specific conclusion}.$$

**Proposition 5.63** (Problem-specific Mirror/Transfer implications are [Spec] by default). *Every problem-specific Mirror/Transfer implication is [Spec] by default unless separately promoted and reproved under the Core guard-rails and the relevant interface protocol.*

*Proof.* By Chapter 8, problem-specific implications do not enlarge the Core by default. Mirror/Transfer implications involving arithmetic, PDE, geometric, mirror-symmetric, categorical, or transfer-theoretic meaning import external semantics. Therefore they are [Spec] unless separately promoted and reproved.  $\square$

**Remark 5.64** (Mirror/Transfer does not solve external problems). A controlled Mirror/Transfer diagram may be useful evidence or structure. It does not by itself prove an arithmetic, PDE, geometric, mirror-symmetric, categorical, or transfer-theoretic theorem. Such conclusions require explicit interface hypotheses and transport theorems.

## L.16. Summary of Mirror/Transfer ledger discipline

**Theorem 5.65** (Appendix L Mirror/Transfer ledger package). *Within the Technical Extension layer, the following package is available.*

- (i) *Mirror and Transfer operations must have declared source, target, realization target, and variance.*
- (ii) *Mirror/Transfer comparison squares compare two declared composites.*
- (iii) *Mirror/Transfer 2-cells witness controlled comparison, not equality, unless zero-cost or strict.*
- (iv) *Nonzero Mirror/Transfer 2-cell costs affect Core audit only through the  $\delta$ -ledger.*
- (v) *Mirror-collapse, Transfer-collapse, Mirror-window, Transfer-window, Mirror-realization, and Transfer-realization comparisons are not automatic.*
- (vi) *Mirror-evaluation and Transfer-evaluation comparisons are not automatic.*
- (vii) *Any nonzero Mirror/Transfer defect used in certification must be budgeted.*
- (viii) *Mirror/Transfer realization comparisons do not replace eligibility.*
- (ix) *Mirror/Transfer diagrams do not replace tower comparison artifacts.*
- (x) *Only strict, zero-cost, or controlled ledger-certified Mirror/Transfer comparisons may support Core certification.*
- (xi) *Mirror/Transfer adjusted budgets must satisfy*

$$\Sigma \delta_{\text{MT}} < \text{gap}_{\tau}.$$

- (xii) *Mirror/Transfer Type IV proxies do not replace monitored Type IV diagnostics.*
- (xiii) *Problem-specific Mirror/Transfer implications are [Spec] by default.*

*Proof.* Items (i)–(iii) follow from the definitions of Mirror/Transfer settings, comparison squares, and 2-cells. Item (iv) is Proposition 5.23. Items (v)–(vii) follow from the collapse, window, realization, and evaluation comparison definitions and the corresponding budget propositions. Item (viii) is Proposition 5.44. Item (ix) is Proposition 5.48. Item (x) is Proposition 5.53. Item (xi) is Proposition 5.56. Item (xii) is Proposition 5.60. Item (xiii) is Proposition 5.63.  $\square$

## L.17. Explicit exclusions

The following are not asserted in Appendix L.

- No Mirror operation is assumed to be a strict equivalence unless explicitly declared and proved.
- No Transfer operation is assumed to preserve Core-relevant data unless explicitly declared and proved.
- No Mirror/Transfer comparison is treated as commuting without strict, zero-cost, or controlled status.
- No nonzero Mirror/Transfer defect is ignored when used in certification.
- No Mirror/Transfer 2-cell replaces B-Gate<sup>+</sup>.
- No Mirror/Transfer 2-cell replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No Mirror/Transfer 2-cell replaces monitored Type IV diagnostics.
- No Mirror/Transfer 2-cell replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No Mirror/Transfer 2-cell replaces the budget condition

$$\Sigma\delta < \text{gap}_\tau.$$

- No Mirror/Transfer realization comparison replaces eligibility.
- No Mirror/Transfer evaluation comparison replaces direct repaired evaluation unless strict, zero-cost, or controlled status is declared.
- No Mirror/Transfer Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No advisory Mirror/Transfer diagram supports Core acceptance.
- No global exactness of  $T_\tau^{\text{bar}}$  is assumed.
- No positive-threshold Serre localization is assumed.
- No global strict filtered lift  $C_\tau$  is assumed.

- No identity

$$C_{\tau,W}F = (C_\tau F)|_W$$

is assumed.

- No problem-specific arithmetic, PDE, geometric, categorical, mirror-symmetric, or transfer-theoretic conclusion is proved here.
- No appendix or companion material enlarges the Core by Mirror/Transfer terminology alone.

## L.18. Provenance and status

The material in this appendix depends on:

- (i) the controlled-commutation framework of Appendix K;
- (ii) the repaired barcode-level collapse and window cautions of Appendix A;
- (iii) the admissible representative discipline of Appendix B;
- (iv) the eligible realization regime of Appendix C;
- (v) the monitored Type IV diagnostics of Appendix D;
- (vi) the audit and ledger discipline of Chapter 6;
- (vii) the interface protocol and [Spec] discipline of Chapter 8;
- (viii) the manifest schema of Appendix G.

Its role is to allow Mirror/Transfer comparisons to be recorded, costed, and audited without allowing them to contaminate Core verdicts.

**Status labels for Appendix L.** *Technical extension definitions:* Definitions [5.4](#), [5.5](#), [5.6](#), [5.8](#), [5.9](#), [5.12](#), [5.13](#), [5.14](#), [5.16](#), [5.17](#), [5.18](#), [5.19](#), [5.21](#), [5.22](#), [5.25](#), [5.26](#), [5.27](#), [5.31](#), [5.32](#), [5.33](#), [5.36](#), [5.37](#), [5.38](#), [5.41](#), [5.42](#), [5.43](#), [5.46](#), [5.47](#), [5.50](#), [5.51](#), [5.52](#), [5.55](#), [5.58](#), [5.59](#), [5.62](#).

*Technical extension propositions/theorems:* Propositions [5.23](#), [5.28](#), [5.35](#), [5.39](#), [5.44](#), [5.48](#), [5.53](#), [5.56](#), [5.60](#), [5.63](#), Theorem [5.65](#).

*Operational cautions:* Remarks [5.1](#), [5.2](#), [5.3](#), [5.7](#), [5.10](#), [5.11](#), [5.15](#), [5.20](#), [5.24](#), [5.29](#), [5.30](#), [5.34](#), [5.40](#), [5.45](#), [5.49](#), [5.54](#), [5.57](#), [5.61](#), [5.64](#).

*Explicit exclusions:* Section L.17.

## 6 Appendix M: Quantale / Ledger Catalog

### M.1. Scope and non-Core status

This appendix records optional quantale and ledger semantics that may be used to instantiate the  $\delta$ -ledger of Chapter 6. It belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

A ledger semantics affects Core audit only after it is explicitly declared in a certificate record.

Thus, this appendix provides a catalog of admissible budget semantics, but it does not choose one globally for all applications. A certified run must declare which budget semantics it uses.

The ledger does not replace Core verification. It only records and aggregates declared discrepancies, costs, tolerances, and controlled deviations that affect the audit condition

$$\Sigma\delta < \text{gap}_\tau.$$

In particular, a ledger policy does not replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \phi_{i,W,\tau}, \quad C_{\tau,W}F.$$

**Remark 6.1** (Technical extension status). The Core requires a declared ledger policy and the strict budget comparison

$$\Sigma\delta < \text{gap}_\tau.$$

This appendix explains possible choices for the value domain and aggregation law of  $\Sigma\delta$ . It does not change the audit discipline itself.

**Remark 6.2** (No hidden default). There is no hidden default ledger semantics. Ordinary addition, maximum aggregation, weighted aggregation, lexicographic aggregation, product ledgers, Boolean ledgers, status ledgers, and hybrid ledgers are different choices. A certificate record must declare which one is being used.

**Remark 6.3** (Relation to Appendix G). Appendix G requires a ledger policy as part of the Minimal Manifest. The present appendix specifies admissible ledger semantics that may instantiate that manifest component.

## M.2. Quantale preliminaries

**Definition 6.4** (Commutative quantale). A *commutative quantale* is a tuple

$$(\mathbb{V}, \leq, \otimes, \mathbb{I}, \bigvee)$$

such that:

- (i)  $(\mathbb{V}, \leq)$  is a complete lattice;
- (ii)  $(\mathbb{V}, \otimes, \mathbb{I})$  is a commutative monoid;
- (iii)  $\otimes$  preserves arbitrary joins in each variable:

$$a \otimes \bigvee_j b_j = \bigvee_j (a \otimes b_j).$$

**Definition 6.5** (Ledger quantale). A *ledger quantale* is a commutative quantale chosen as the value domain for budget components in an AK certificate record. Its monoidal operation is used to aggregate budget contributions.

**Definition 6.6** (Budget order). The *budget order* is the partial order

$$\leq$$

on the ledger quantale. It determines when one budget value is no larger than another and therefore supports comparisons such as

$$\Sigma\delta < \text{gap}_\tau.$$

**Definition 6.7** (Strict budget comparison). Given a ledger order  $\leq$ , the strict comparison

$$a < b$$

means

$$a \leq b \quad \text{and} \quad a \text{ eq } b,$$

unless a different strict-order convention is explicitly declared and shown compatible with the chosen ledger semantics.



**Remark 6.8** (Strict comparison). The Core audit condition uses a strict comparison:

$$\Sigma\delta < \text{gap}_\tau.$$

The boundary case

$$\Sigma\delta = \text{gap}_\tau$$

is not an audited pass.

**Definition 6.9** (Ledger policy). A *ledger policy* consists of:

- (i) a ledger value domain  $V$ ;
- (ii) an order  $\leq$ ;
- (iii) an aggregation law  $\otimes$ ;
- (iv) a unit  $\mathbb{I}$ ;
- (v) a strict comparison convention;
- (vi) the declared list of components;
- (vii) the safety margin  $\text{gap}_\tau$ ;
- (viii) artifact references supporting component values and aggregation.

### M.3. Ledger components

**Definition 6.10** (Ledger component). A *ledger component* is a declared budget contribution

$$\delta^\alpha \in V$$

associated with a specific source of deviation, approximation, uncertainty, controlled comparison, representative discrepancy, evaluation discrepancy, or artifact transport cost.

**Definition 6.11** (Standard ledger components). The following component classes are standard in the AK framework:

- (i)  $\delta^{\text{eval}}$ : repaired evaluation-route discrepancy, including deviations affecting

$$\text{Eval}_{i,W,\tau}(F);$$

- (ii)  $\delta^{\text{rep}}$ : admissible representative discrepancy associated with

$$C_{\tau,W}F;$$

- (iii)  $\delta^{\text{elig}}$ : eligibility or realization-check discrepancy when the  $\text{Ext}^1$ -clause is invoked;

- (iv)  $\delta^\phi$ : tower comparison artifact discrepancy associated with

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty);$$

- (v)  $\delta^{\text{alg}}$ : algorithmic or implementation discrepancy;

- (vi)  $\delta^{\text{disc}}$ : discretization or grid-to-continuum discrepancy;
- (vii)  $\delta^{\text{meas}}$ : measurement or observation discrepancy;
- (viii)  $\delta^{\text{comm}}$ : controlled-commutation discrepancy;
- (ix)  $\delta^{2\text{-cell}}$ : controlled 2-cell cost;
- (x)  $\delta^{\text{MT}}$ : Mirror/Transfer comparison cost;
- (xi)  $\delta^{\text{ov}}$ : overlap compatibility discrepancy;
- (xii)  $\delta^{\text{rst}}$ : restart or continuation discrepancy;
- (xiii)  $\delta^{\text{adv}}$ : advisory quantity converted into a declared budget contribution;
- (xiv)  $\delta^{\text{src}}$ : source-data or import discrepancy;
- (xv)  $\delta^{\text{ser}}$ : serialization, parsing, or execution-schema discrepancy when relevant.

**Definition 6.12** (Total ledger). Given ledger components

$$\delta^1, \dots, \delta^m \in \mathbb{V},$$

the total ledger is the declared aggregate

$$\Sigma\delta := \delta^1 \otimes \dots \otimes \delta^m,$$

or the corresponding declared finite, countable, sequential, parallel, windowwise, overlapwise, or product aggregation specified by the ledger policy.

**Definition 6.13** (Component support record). A *component support record* for  $\delta^\alpha$  contains:

- (i) the source of the component;
- (ii) the value of the component;
- (iii) the units or type of the value;
- (iv) the artifact references supporting the value;
- (v) the rule by which the component enters  $\Sigma\delta$ ;
- (vi) whether the component is verifier-side, advisory-converted, or technical-extension data.

**Remark 6.14** (Components must be typed). Every component must declare its source, units or type, aggregation rule, and supporting artifact references. Untyped numerical error terms are not audit-readable.

**Proposition 6.15** (Missing component typing defeats auditability). *If a ledger component is used in a certificate but its type, source, aggregation rule, or supporting artifacts are missing, then the budget comparison is not audit-readable.*

*Proof.* The verifier must reconstruct both the value of the component and how it contributes to

$$\Sigma\delta.$$

Without type, source, aggregation, and artifact support, this reconstruction is impossible. Therefore the corresponding budget comparison is not audit-readable.  $\square$

## M.4. Additive ledger

**Definition 6.16** (Additive ledger). The *additive ledger* is the ledger semantics

$$\mathbb{V} = [0, \infty], \quad \otimes = +, \quad \mathbb{I} = 0, \quad \leq = \text{usual order}.$$

In this semantics,

$$\Sigma\delta = \sum_{\alpha} \delta^{\alpha}.$$

**Proposition 6.17** (Additive budget pass). *Under the additive ledger, Audit Mode budget admissibility is:*

$$\sum_{\alpha} \delta^{\alpha} < \text{gap}_{\tau}.$$

*Proof.* This is the strict Core budget condition specialized to the additive monoidal operation.  $\square$

**Remark 6.18** (When additive ledger is appropriate). The additive ledger is appropriate when independent or sequential deviations accumulate by ordinary summation. It is often natural for discretization, measurement, algorithmic error, and sequential controlled-commutation budgets.

## M.5. Max ledger

**Definition 6.19** (Max ledger). The *max ledger* is the ledger semantics

$$\mathbb{V} = [0, \infty], \quad \otimes = \max, \quad \mathbb{I} = 0, \quad \leq = \text{usual order}.$$

In this semantics,

$$\Sigma\delta = \max_{\alpha} \delta^{\alpha}.$$

**Proposition 6.20** (Max budget pass). *Under the max ledger, Audit Mode budget admissibility is:*

$$\max_{\alpha} \delta^{\alpha} < \text{gap}_{\tau}.$$

*Proof.* This is the strict Core budget condition specialized to the max monoidal operation.  $\square$

**Remark 6.21** (When max ledger is appropriate). The max ledger is appropriate when deviations are not additive but instead bounded by the worst active component. It is useful for sup-norm style controls, uniform bounds, or parallel checks where the largest discrepancy dominates.

## M.6. Weighted additive ledger

**Definition 6.22** (Weighted additive ledger). A *weighted additive ledger* is a ledger semantics of the form

$$\Sigma\delta = \sum_{\alpha} w_{\alpha} \delta^{\alpha},$$

where

$$w_{\alpha} \geq 0$$

are declared weights.

**Definition 6.23** (Weight policy). A *weight policy* specifies:

(i) the components being weighted;

- (ii) the weights  $w_\alpha$ ;
- (iii) the reason for each weight;
- (iv) whether the weights are fixed, learned, calibrated, or externally declared;
- (v) artifact references supporting the policy;
- (vi) whether the policy is verifier-side or advisory.

**Proposition 6.24** (Weighted budget pass). *Under a weighted additive ledger, budget admissibility is:*

$$\sum_{\alpha} w_{\alpha} \delta^{\alpha} < \text{gap}_{\tau}.$$

*Proof.* This follows from the definition of the weighted additive ledger and the strict Core budget comparison.  $\square$

**Remark 6.25** (Weight policies are sensitive). Weights can change the audit outcome. Therefore any weighted ledger must be declared explicitly. Hidden, learned-after-the-fact, or post-hoc weight choices are not admissible.

## M.7. Lexicographic ledger

**Definition 6.26** (Lexicographic ledger). A *lexicographic ledger* takes values in

$$\mathbf{V} = [0, \infty]^r$$

with lexicographic order. A budget vector is written

$$\delta = (\delta_1, \dots, \delta_r).$$

Aggregation is componentwise:

$$\delta \otimes \epsilon = (\delta_1 + \epsilon_1, \dots, \delta_r + \epsilon_r),$$

or another declared componentwise law.

**Definition 6.27** (Lexicographic priority). The *lexicographic priority order* declares that the first coordinate dominates the second, the second dominates the third, and so on. Thus reducing a higher-priority coordinate is more important than any change in lower-priority coordinates.

**Proposition 6.28** (Lexicographic budget pass). *Under a lexicographic ledger, budget admissibility is:*

$$\Sigma \delta <_{\text{lex}} \text{gap}_{\tau}.$$

*Proof.* This is the strict Core budget comparison interpreted in the lexicographic order.  $\square$

**Remark 6.29** (When lexicographic ledger is appropriate). A lexicographic ledger is appropriate when some error classes are categorically more important than others. For example, one may place structural defects, eligibility defects, or tower-artifact defects before measurement noise or advisory-converted quantities.

## M.8. Product ledger

**Definition 6.30** (Product ledger). Let

$$(V_1, \otimes_1, \mathbb{I}_1, \leq_1), \dots, (V_r, \otimes_r, \mathbb{I}_r, \leq_r)$$

be ledger quantales. Their *product ledger* is

$$V = V_1 \times \dots \times V_r$$

with componentwise aggregation:

$$(v_1, \dots, v_r) \otimes (w_1, \dots, w_r) = (v_1 \otimes_1 w_1, \dots, v_r \otimes_r w_r),$$

and componentwise order unless another order is declared.

**Definition 6.31** (Product gap). A *product gap* is a tuple

$$\text{gap}_\tau = (\text{gap}_{\tau,1}, \dots, \text{gap}_{\tau,r})$$

against which a product ledger budget is compared.

**Proposition 6.32** (Product budget pass). *Under the componentwise product ledger, budget admissibility is:*

$$\Sigma \delta_j < \text{gap}_{\tau,j} \quad \text{for every coordinate } j.$$

*Proof.* This is the strict Core budget condition interpreted componentwise. □

**Remark 6.33** (When product ledger is appropriate). A product ledger is useful when different error classes should not be collapsed into a single scalar. For example, one may separately track repaired-evaluation discrepancy, representative discrepancy, discretization error, measurement error, commutation defect, and Mirror/Transfer cost.

## M.9. Boolean and status ledgers

**Definition 6.34** (Boolean ledger). The *Boolean ledger* has values

$$V = \{\text{pass}, \text{fail}\}$$

with aggregation

$$\text{pass} \otimes \text{pass} = \text{pass},$$

and any aggregation involving

$$\text{fail}$$

equal to

$$\text{fail}.$$

**Definition 6.35** (Status ledger). A *status ledger* records qualitative states such as:

$$\text{strict}, \quad \text{zero-cost}, \quad \text{controlled}, \quad \text{advisory}, \quad \text{uncontrolled}, \quad \text{failed}.$$

Its aggregation law must be declared explicitly.

**Definition 6.36** (Core-admissible status class). A status class is *Core-admissible* if it is explicitly declared to support certificate use. Typical admissible statuses are:

strict,      zero-cost,      controlled.

Typical non-admissible statuses are:

advisory,      uncontrolled,      failed.

**Remark 6.37** (Qualitative ledgers). Boolean and status ledgers are useful for categorical flags, availability checks, artifact status, or qualitative workflow states. They are insufficient for numerical budget comparison unless paired with a numerical or ordered ledger.

## M.10. Hybrid ledger

**Definition 6.38** (Hybrid ledger). A *hybrid ledger* combines numerical and qualitative components, for example:

$$\mathbb{V} = [0, \infty] \times \{\text{strict, zero-cost, controlled, advisory, uncontrolled, failed}\}.$$

Aggregation is defined by a declared numerical law together with a declared status law.

**Definition 6.39** (Hybrid admissibility). A hybrid ledger passes if:

- (i) the numerical component satisfies the strict budget condition;
- (ii) the qualitative component lies in a declared Core-admissible status class.

**Proposition 6.40** (Hybrid pass condition). *Under a hybrid ledger, Audit Mode may pass only if both the numerical and qualitative admissibility conditions pass.*

*Proof.* The hybrid ledger contains multiple components of audit information. Failure of either the numerical or qualitative component means the ledger as a whole is not admissible.  $\square$

**Remark 6.41** (Hybrid ledgers and controlled commutation). Hybrid ledgers are useful when controlled commutation, Mirror/Transfer data, representative construction, or tower artifact transport have both a numerical defect and a status classification such as strict, zero-cost, controlled, advisory, uncontrolled, or failed.

## M.11. Ledger conversion

**Definition 6.42** (Ledger conversion). A *ledger conversion* is a declared map

$$\kappa : \mathbb{V} \rightarrow \mathbb{W}$$

from one ledger semantics to another.

**Definition 6.43** (Admissible ledger conversion). A ledger conversion

$$\kappa : \mathbb{V} \rightarrow \mathbb{W}$$

is *admissible* if:

- (i) it is monotone:

$$v \leq_{\mathbb{V}} v' \Rightarrow \kappa(v) \leq_{\mathbb{W}} \kappa(v');$$

(ii) it respects aggregation up to a declared inequality:

$$\kappa(v \otimes_V v') \leq_W \kappa(v) \otimes_W \kappa(v');$$

(iii) it preserves or tightens budget admissibility;

(iv) it is declared in the certificate record;

(v) it includes artifact references or proof references supporting the conversion.

**Proposition 6.44** (Ledger conversion cannot loosen audit without declaration). *A ledger conversion may not be used to turn a failing budget into a passing budget unless the conversion is explicitly declared and proved to preserve the intended audit semantics.*

*Proof.* A conversion that changes pass/fail status changes the audit semantics. By Core audit discipline, such semantics must be declared before evaluation. Post-hoc conversion is not audit-readable and cannot justify acceptance.  $\square$

**Remark 6.45** (Conversion risk). Ledger conversion is useful but dangerous. For example, converting a product ledger to a scalar ledger may hide which coordinate failed. Such conversions must be conservative or explicitly justified.

## M.12. Ledger refinement

**Definition 6.46** (Ledger refinement). *A ledger refinement replaces a coarse budget component*

$$\delta^\alpha$$

by a family of finer components

$$\delta^{\alpha_1}, \dots, \delta^{\alpha_m}$$

together with an aggregation law satisfying

$$\delta^\alpha \succeq \delta^{\alpha_1} \otimes \dots \otimes \delta^{\alpha_m}$$

in the declared order.

**Proposition 6.47** (Refinement may improve audit transparency). *A ledger refinement improves audit transparency when it preserves or tightens the total budget and exposes the sources of error more explicitly.*

*Proof.* Refinement decomposes a coarse component into traceable subcomponents. If aggregation of the refined components is bounded by the original component, the total budget is not loosened. Thus the certificate becomes more transparent without weakening the audit condition.  $\square$

**Remark 6.48** (No post-hoc refinement to force pass). A refinement may not be introduced after failure merely to force a pass. The refinement policy must be declared before or as part of the certificate record.

### M.13. Ledger compatibility with repaired evaluation

**Definition 6.49** (Evaluation ledger). An *evaluation ledger* records discrepancies affecting the repaired profile

$$\text{Eval}_{i,W,\tau}(F) = T_{\tau}^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

Its standard component is

$$\delta^{\text{eval}}.$$

**Definition 6.50** (Evaluation-route discrepancy). An *evaluation-route discrepancy* is a declared discrepancy between:

$$\text{Eval}_{i,W,\tau}(F)$$

and an implemented, approximated, transported, discretized, or alternate-route profile used in a certificate record.

**Proposition 6.51** (Evaluation-route discrepancies must be ledgered). *If a certificate relies on an implemented or alternate repaired-evaluation route rather than a direct verified evaluation of*

$$\text{Eval}_{i,W,\tau}(F),$$

*then the evaluation-route discrepancy must be included in the ledger.*

*Proof.* The repaired evaluation profile is a verifier-side object. Any discrepancy between the ideal repaired profile and the implemented or alternate profile changes the evidence used by the Core. Therefore the discrepancy must contribute to  $\Sigma\delta$ .  $\square$

**Remark 6.52** (No ledger replacement of evaluation). A ledger component may bound discrepancy in an evaluation route. It does not replace the repaired evaluation record itself. The certificate must still declare the profile or the artifact path by which it is reconstructed.

### M.14. Ledger compatibility with representatives and eligibility

**Definition 6.53** (Representative ledger). A *representative ledger* records discrepancies associated with admissible representatives

$$C_{\tau,W}F.$$

Its standard component is

$$\delta^{\text{rep}}.$$

**Definition 6.54** (Eligibility ledger). An *eligibility ledger* records declared discrepancy or tolerance associated with verifying:

$$\mathcal{R}(C_{\tau,W}F) \in D^{[-1,0]}(k\text{-mod})$$

and the edge-realization condition required for the  $\text{Ext}^1$ -clause. Its standard component is

$$\delta^{\text{elig}}.$$

**Proposition 6.55** (Representative ledger does not replace admissibility). *A representative ledger component does not replace the requirement that*

$$C_{\tau,W}F$$

*be an admissible representative of*

$$\text{Eval}_{i,W,\tau}(F)$$

*in the monitored degrees.*



*Proof.* Admissibility is a structural compatibility condition. A ledger component may record discrepancy or cost associated with representative construction, but it cannot turn a non-admissible representative into an admissible one.  $\square$

**Proposition 6.56** (Eligibility ledger does not replace eligibility). *An eligibility ledger component does not replace the eligible realization requirements of Chapter 3 and Appendix C.*

*Proof.* The  $\text{Ext}^1$ -clause requires amplitude  $[-1, 0]$  and edge realization. A budget component may record tolerance or verification cost, but it does not itself prove those conditions.  $\square$

## M.15. Ledger compatibility with tower comparison artifacts

**Definition 6.57** (Tower artifact ledger). *A tower artifact ledger records discrepancies associated with the tower comparison artifact*

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty).$$

Its standard component is

$$\delta^\phi.$$

**Definition 6.58** (Tower artifact transport discrepancy). *A tower artifact transport discrepancy is a declared discrepancy introduced when a tower comparison artifact is approximated, transported, converted, discretized, mirrored, transferred, or reconstructed from another artifact.*

**Proposition 6.59** (Tower artifact ledger does not replace the artifact). *A tower artifact ledger component does not replace the required tower comparison artifact*

$$\phi_{i,W,\tau}.$$

*If Type IV diagnostics are invoked, the artifact and its kernel/cokernel defect data must still be declared.*

*Proof.* The diagnostics

$$\mu_{\text{Collapse}}, u_{\text{Collapse}}$$

are computed from the kernel and cokernel defect profiles of

$$\phi_{i,W,\tau}.$$

A ledger component may account for discrepancy in obtaining or transporting that artifact, but without the artifact the diagnostics are not defined.  $\square$

**Remark 6.60** (Artifact cost vs. diagnostic value). The cost of constructing, transporting, or approximating a tower comparison artifact is different from the diagnostic values

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

Both may appear in a certificate record, but they play different roles.

## M.16. Ledger compatibility with windows

**Definition 6.61** (Windowwise ledger). A *windowwise ledger* assigns a budget

$$\Sigma\delta_W$$

to each declared window  $W$ .

**Definition 6.62** (Global window ledger). For a family of windows  $\{W_\alpha\}$ , the *global window ledger* is the declared aggregation

$$\Sigma\delta_{\text{win}} = \bigotimes_{\alpha} \Sigma\delta_{W_\alpha}.$$

**Proposition 6.63** (Windowwise pass does not automatically imply global pass). *Even if*

$$\Sigma\delta_{W_\alpha} < \text{gap}_{\tau, \alpha}$$

*for every window  $W_\alpha$ , global budget admissibility does not follow unless the global aggregation law and global gap comparison are declared and satisfied.*

*Proof.* Windowwise inequalities are local. A global certificate requires a global or assembled budget comparison. Without declaring how local budgets aggregate and what global gap applies, the global pass condition is not determined.  $\square$

**Remark 6.64** (Relation to Appendix E). Appendix E handles window, overlap, restart, and summability logic. This appendix supplies the budget semantics used by those constructions.

## M.17. Ledger compatibility with overlaps

**Definition 6.65** (Overlap ledger). An *overlap ledger* records the overlap budgets

$$\Sigma\delta_{\alpha\beta}$$

used in Overlap Gate checks.

**Definition 6.66** (Total overlap budget). The *total overlap budget* is the declared aggregation

$$\Sigma\delta_{\text{ov}} = \bigotimes_{\alpha, \beta} \Sigma\delta_{\alpha\beta},$$

over declared nonempty overlaps.

**Proposition 6.67** (Overlap ledger must match Overlap Gate). *The overlap ledger is admissible only if its components match the overlap budgets used in the Overlap Gate verdicts.*

*Proof.* The Overlap Gate uses specific budgets to judge compatibility. If the ledger records different budgets, the certificate record is not reconstructible. Thus the ledger and gate verdicts must match.  $\square$

## M.18. Ledger compatibility with restart and summability

**Definition 6.68** (Restart ledger). A *restart ledger* records the per-window continuation or restart costs

$$\delta_k^{\text{rst}}$$

used when passing from one MECE window  $W_k$  to the next.

**Definition 6.69** (Summable ledger sequence). A sequence of ledger values

$$\{\delta_k\}_{k \geq 0}$$

is *summable* if its declared aggregate

$$\bigotimes_{k \geq 0} \delta_k$$

exists and remains below the declared global or windowwise gap according to the chosen ledger semantics.

**Proposition 6.70** (Summability depends on ledger semantics). *Whether a sequence of budgets is summable depends on the chosen ledger quantale and aggregation law.*

*Proof.* A sequence that is summable under a max ledger may fail to be summable under an additive ledger. Similarly, a sequence may pass in one coordinate of a product ledger but fail in another. Therefore summability is not an absolute property; it is ledger-relative.  $\square$

**Remark 6.71** (No ledger-free summability). Statements such as

$$\sum_k \delta_k < \infty$$

are meaningful only after the aggregation semantics are declared.

## M.19. Ledger compatibility with advisory quantities

**Definition 6.72** (Advisory-to-budget conversion). An *advisory-to-budget conversion* is a declared rule assigning a budget component

$$\delta^{\text{adv}}$$

to an advisory invariant, spectral indicator, warning flag, navigation quantity, proxy, or candidate score.

**Proposition 6.73** (Advisory quantities are budget-neutral unless converted). *An advisory quantity has no effect on the Core budget comparison unless an advisory-to-budget conversion is explicitly declared.*

*Proof.* The Core budget comparison uses only declared ledger components. An advisory quantity is not a budget component by default. Therefore it is budget-neutral unless converted.  $\square$

**Remark 6.74** (Relation to Appendix H). Appendix H defines advisory invariants and navigation quantities. The present appendix specifies how, if desired, such quantities may be converted into declared budget components.

## M.20. Ledger compatibility with discretization and measurement

**Definition 6.75** (Discretization ledger). A *discretization ledger* records discretization or grid-to-continuum error components

$$\delta^{\text{disc}}.$$

**Definition 6.76** (Measurement ledger). A *measurement ledger* records measurement, observation, sampling, or aggregate-estimation error components

$$\delta^{\text{meas}}.$$

**Definition 6.77** (Combined discretization-measurement ledger). A combined discretization-measurement ledger aggregates

$$\delta^{\text{disc}} \quad \text{and} \quad \delta^{\text{meas}}$$

according to the declared ledger semantics.

**Proposition 6.78** (Discretization and measurement errors must aggregate consistently). *Discretization and measurement errors are audit-admissible only if they are aggregated according to the declared ledger semantics.*

*Proof.* Both  $\delta^{\text{disc}}$  and  $\delta^{\text{meas}}$  contribute to the total budget when discretized or measured evidence is used. Therefore their aggregation must follow the ledger policy used to compute  $\Sigma\delta$ .  $\square$

## M.21. Ledger compatibility with controlled commutation and Mirror/Transfer

**Definition 6.79** (Controlled-comparison ledger). A *controlled-comparison ledger* records defects from controlled commutation, AWFS-assisted comparison, or 2-cell comparison:

$$\delta^{\text{comm}}, \quad \delta^{\text{2-cell}}.$$

**Definition 6.80** (Mirror/Transfer ledger component). A *Mirror/Transfer ledger component* records the aggregate cost

$$\delta^{\text{MT}}$$

of Mirror/Transfer comparisons used in a certificate record.

**Proposition 6.81** (Controlled comparison costs must aggregate consistently). *Controlled commutation and Mirror/Transfer costs are audit-admissible only if they are aggregated according to the declared ledger semantics.*

*Proof.* Controlled comparison costs contribute to  $\Sigma\delta$ . If their aggregation does not follow the declared ledger semantics, then the total budget is not reconstructible. Thus the aggregation must be consistent with the ledger policy.  $\square$

**Remark 6.82** (Relation to Appendices K and L). Appendix K defines controlled commutation defects. Appendix L defines Mirror/Transfer 2-cell costs. Appendix M specifies the ledger semantics by which those costs are aggregated.

## M.22. Ledger manifest requirements

**Definition 6.83** (Ledger manifest entry). A *ledger manifest entry* records:

- (i) the ledger quantale or ledger value domain;
- (ii) the aggregation law;
- (iii) the strict order convention;
- (iv) all ledger components;
- (v) the component support records;
- (vi) the total budget  $\Sigma\delta$ ;
- (vii) the safety margin  $\text{gap}_\tau$ ;
- (viii) the pass/fail result of the budget comparison;
- (ix) artifact references supporting all component values;
- (x) any ledger conversion, refinement, or coordinate projection used.

**Proposition 6.84** (Ledger manifest is required for audit readability). *A certificate record is not audit-readable unless its ledger semantics and ledger components are declared.*

*Proof.* Audit Mode requires the strict comparison

$$\Sigma\delta < \text{gap}_\tau.$$

Without the ledger semantics and component values,  $\Sigma\delta$  cannot be reconstructed. Thus the certificate is not audit-readable.  $\square$

**Remark 6.85** (Ledger manifest and Appendix G). Appendix G requires a ledger policy as part of the Minimal Manifest. This appendix specifies the technical content that such a ledger policy may contain.

## M.23. Ledger non-monotonicity warnings

**Definition 6.86** (Non-monotone ledger transformation). A ledger transformation

$$\kappa : V \rightarrow W$$

is *non-monotone* if there exist  $a \leq b$  in  $V$  such that

$$\kappa(a) \not\leq \kappa(b)$$

in  $W$ .

**Proposition 6.87** (Non-monotone transformations are not audit-admissible by default). *A non-monotone ledger transformation is not audit-admissible unless a separate justification is declared and proved.*

*Proof.* Budget audit depends on order comparison. A non-monotone transformation can reverse or obscure budget ordering. Therefore it cannot be used safely without additional justification.  $\square$

**Remark 6.88** (Why monotonicity matters). If a larger error can be transformed into a smaller budget value, then the ledger can hide risk. Monotonicity prevents this kind of audit failure.

## M.24. Ledger cannot replace Core gates

**Proposition 6.89** (Budget pass does not imply B-Gate<sup>+</sup> pass). *The condition*

$$\Sigma\delta < \text{gap}_\tau$$

*does not imply B-Gate<sup>+</sup> pass.*

*Proof.* B-Gate<sup>+</sup> also requires repaired topological vanishing, eligible categorical vanishing when invoked, monitored Type IV diagnostic vanishing, and required artifact declarations. The budget condition is necessary for audited acceptance but is not the whole gate.  $\square$

**Proposition 6.90** (B-Gate<sup>+</sup> structural pass does not imply audit pass without ledger). *Even if the structural clauses of B-Gate<sup>+</sup> pass, Audit Mode does not pass unless the ledger comparison*

$$\Sigma\delta < \text{gap}_\tau$$

*is declared and satisfied.*

*Proof.* Chapter 6 separates structural gate pass from audited pass. Audit Mode requires the strict budget comparison in addition to structural gate conditions.  $\square$

**Remark 6.91** (Ledger and gate are complementary). The gate checks whether the repaired Core conditions hold. The ledger checks whether declared deviations remain within the safety margin. Neither component replaces the other.

## M.25. Summary of quantale / ledger catalog

**Theorem 6.92** (Appendix M quantale-ledger package). *Within the Technical Extension layer, the following package is available.*

- (i) *A ledger policy must declare a value domain, aggregation law, order, strict comparison convention, components, total budget, and gap comparison.*
- (ii) *Commutative quantales provide a general semantics for budget aggregation.*
- (iii) *Additive, max, weighted additive, lexicographic, product, Boolean, status, and hybrid ledgers are admissible only when explicitly declared.*
- (iv) *Ledger components must be typed and artifact-supported.*
- (v) *Standard components include evaluation-route, representative, eligibility, tower-artifact, algorithmic, discretization, measurement, controlled-commutation, 2-cell, Mirror/Transfer, overlap, restart, advisory-converted, source-data, and serialization components.*
- (vi) *Weighted ledgers require declared weight policies.*
- (vii) *Product ledgers preserve separate error coordinates.*
- (viii) *Hybrid ledgers combine numerical and qualitative status conditions.*
- (ix) *Ledger conversion must be monotone and audit-preserving.*
- (x) *Ledger refinement may improve transparency but may not be used post-hoc to force acceptance.*

- (xi) *Evaluation-route discrepancies, representative discrepancies, eligibility discrepancies, and tower-artifact discrepancies must be ledgered when used.*
- (xii) *Window, overlap, restart, summability, advisory, discretization, measurement, commutation, and Mirror/Transfer costs must aggregate according to the declared ledger semantics.*
- (xiii) *Advisory quantities are budget-neutral unless explicitly converted.*
- (xiv) *Non-monotone ledger transformations are not audit-admissible by default.*
- (xv) *A ledger manifest entry is required for audit readability.*
- (xvi) *Budget pass does not replace B-Gate<sup>+</sup>, and structural B-Gate<sup>+</sup> pass does not replace the ledger.*

*Proof.* Items (i)–(iii) follow from the definitions of ledger policies, ledger quantales, and cataloged ledger types. Items (iv)–(v) follow from typed component and standard component requirements. Item (vi) follows from the weighted ledger policy. Items (vii)–(viii) follow from the definitions of product and hybrid ledgers. Item (ix) is the admissible ledger conversion condition. Item (x) follows from the refinement discipline. Item (xi) follows from Sections M.13–M.15. Item (xii) follows from compatibility with Appendices E, I, J, K, and L. Item (xiii) follows from the advisory-to-budget conversion rule. Item (xiv) follows from Proposition 6.87. Item (xv) follows from the ledger manifest requirement. Item (xvi) follows from Propositions 6.89 and 6.90.  $\square$

## M.26. Explicit exclusions

The following are not asserted in Appendix M.

- No single global ledger semantics is imposed on all certificates.
- No hidden default aggregation law is assumed.
- No ordinary addition is assumed unless declared.
- No weight policy is accepted without declaration.
- No product ledger is collapsed to a scalar ledger without declared conversion.
- No advisory quantity affects budget unless converted.
- No repaired evaluation route discrepancy is ignored when used in certification.
- No representative discrepancy is allowed to replace representative admissibility.
- No eligibility ledger component replaces eligibility.
- No tower-artifact cost replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No controlled-commutation defect is ignored when used in certification.
- No Mirror/Transfer cost is ignored when used in certification.
- No non-monotone ledger conversion is accepted by default.

- No post-hoc ledger conversion may turn failure into pass.
- No ledger refinement may be used merely to force acceptance.
- No budget comparison replaces B-Gate<sup>+</sup>.
- No budget comparison replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No budget comparison replaces monitored Type IV diagnostics.
- No budget comparison replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No ledger catalog entry proves a problem-specific arithmetic, PDE, geometric, categorical, mirror-symmetric, or transfer-theoretic result.

## M.27. Provenance and status

The material in this appendix depends on:

- (i) the audit and budget discipline of Chapter 6;
- (ii) the manifest requirements of Appendix G;
- (iii) the repaired evaluation discipline of Appendix A;
- (iv) the admissible representative discipline of Appendix B;
- (v) the eligible realization regime of Appendix C;
- (vi) the tower comparison artifact discipline of Appendix D;
- (vii) the window and overlap budget structures of Appendix E;
- (viii) the advisory-to-budget conversion discipline of Appendix H;
- (ix) the discretization and measurement ledgers of Appendices I and J;
- (x) the controlled-commutation and Mirror/Transfer ledgers of Appendices K and L.

Its role is to provide a catalog of admissible budget semantics without allowing any ledger choice to bypass Core audit.



**Status labels for Appendix M.** *Technical extension definitions:* Definitions 6.4, 6.5, 6.6, 6.7, 6.9, 6.10, 6.11, 6.12, 6.13, 6.16, 6.19, 6.22, 6.23, 6.26, 6.27, 6.30, 6.31, 6.34, 6.35, 6.36, 6.38, 6.39, 6.42, 6.43, 6.46, 6.49, 6.50, 6.53, 6.54, 6.57, 6.58, 6.61, 6.62, 6.65, 6.66, 6.68, 6.69, 6.72, 6.75, 6.76, 6.77, 6.79, 6.80, 6.83, 6.86.

*Technical extension propositions/theorems:* Propositions 6.15, 6.17, 6.20, 6.24, 6.28, 6.32, 6.40, 6.44, 6.47, 6.51, 6.55, 6.56, 6.59, 6.63, 6.67, 6.70, 6.73, 6.78, 6.81, 6.84, 6.87, 6.89, 6.90, Theorem 6.92.

*Operational cautions:* Remarks 6.1, 6.2, 6.3, 6.8, 6.14, 6.18, 6.21, 6.25, 6.29, 6.33, 6.37, 6.41, 6.45, 6.48, 6.52, 6.60, 6.64, 6.71, 6.74, 6.82, 6.85, 6.88, 6.91.

*Explicit exclusions:* Section M.26.

## 7 Appendix N: Derived, Sheaf, and Higher-Categorical Extensions

### N.1. Scope and non-Core status

This appendix records optional derived, sheaf-theoretic, stack-theoretic, and higher-categorical extensions of the AK framework. It belongs to the Technical Extension layer. It does not enlarge the Core mathematical package of Chapters 1–8.

The central rule of this appendix is:

Higher-categorical enrichment does not upgrade Core claims unless explicitly imported and reproved.

Thus, derived categories, sheaf categories, stacks,  $\infty$ -categories, enhancement data, descent structures, and higher functorial comparisons may provide powerful external organization, but they do not replace:

$$B\text{-Gate}^+, \quad \text{Eval}_{1,W,\tau}(F) = 0, \quad (\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0), \quad \phi_{i,W,\tau}, \quad \Sigma\delta < \text{gap}_\tau.$$

All Core-relevant persistence data remain governed by the repaired windowed evaluation:

$$\text{Eval}_{i,W,\tau}(F) = T_\tau^{\text{bar}}(W_W \mathbf{P}_i(F)).$$

**Remark 7.1** (Technical extension status). The Core of the present text is intentionally restricted to the constructible one-parameter persistence regime over a field, admissible representatives in the implementable range, the eligible amplitude- $[-1, 0]$  realization regime, monitored Type IV diagnostics, and audit discipline. The extensions discussed here are optional external enrichments. They are not part of the minimal theorem-bearing Core.

**Remark 7.2** (Relation to Chapter 8). Chapter 8 fixed the protocol for connecting external domains to the Core without upgrading Core claims. The present appendix applies that discipline to derived, sheaf, stack, and higher-categorical settings. Every such extension requires a declared realization target, a route to repaired Core evaluation, an eligible regime when  $\text{Ext}^1$  is invoked, declared tower comparison artifacts when Type IV diagnostics are invoked, and default [Spec] status for problem-specific implications.

**Remark 7.3** (Relation to Appendix G). Appendix G requires that all verifier-side data be manifest-readable. Therefore any derived, sheaf, stack, or higher-categorical enhancement used in certification must provide artifact references, realization routes, ledger entries, and status labels. Otherwise it remains non-verdict technical context.

## N.2. External enhancement principle

**Definition 7.4** (External enhancement). An *external enhancement* of the AK framework is a mathematical structure placed around the Core to organize, compare, refine, or interpret Core-relevant objects without changing the Core verdict rule. Examples include:

- (i) derived enhancements;
- (ii) sheaf-theoretic enhancements;
- (iii) stack-theoretic enhancements;
- (iv) dg-categorical enhancements;
- (v) stable  $\infty$ -categorical enhancements;
- (vi) higher topos-theoretic enhancements;
- (vii) enriched or indexed categorical formulations.

**Definition 7.5** (External enhancement principle). The *external enhancement principle* states that an enhancement may organize or refine the surrounding theory, but it does not alter Core pass/reject unless its output is re-entered into the Core verifier through declared objects, windows, thresholds, repaired evaluation records, admissible representatives when used, eligibility records when invoked, tower comparison artifacts when Type IV is invoked, ledgers, diagnostics, gate verdicts, and artifact references.

**Definition 7.6** (Verifier-side re-entry of an enhancement). An external enhancement  $\mathcal{E}$  *re-enters verifier-side Core data* only if it provides one or more of:

- (i) a Core-readable filtered or persistence object;
- (ii) repaired evaluation data

$$\text{Eval}_{i,w,\tau}(F);$$

- (iii) an admissible representative

$$C_{\tau,w}F;$$

- (iv) an eligible realized object for the  $\text{Ext}^1$ -clause;

- (v) a tower comparison artifact

$$\phi_{i,w,\tau} : \text{ColimProfile}_{i,w,\tau}(F_\bullet) \rightarrow \text{ApexProfile}_{i,w,\tau}(F_\infty);$$

- (vi) a declared diagnostic record;
- (vii) a declared ledger component;
- (viii) a gate verdict record.

**Proposition 7.7** (Enhancement does not update Core verdicts). *Let  $\mathcal{E}$  be any external enhancement introduced in this appendix. Then  $\mathcal{E}$  does not update the Core verdict of a certificate record unless its relevant data are explicitly realized as verifier-side Core data.*

*Proof.* By Chapter 6 and Appendix G, Core verdicts are determined only by verifier-side data: repaired evaluation records, eligibility records when invoked, monitored diagnostics, declared tower comparison artifacts, gate verdicts, artifact references, and the budget comparison. External enhancements are not, by themselves, any of these verifier-side objects. Therefore they do not update Core verdicts unless translated into verifier-side data.  $\square$

**Remark 7.8** (Enhancement vs. proof). An enhancement may make a construction clearer, more functorial, more natural, or more conceptually unified. That does not make it a proof of a Core theorem unless the relevant statement is proved under the Core guard-rails.

### N.3. Derived extensions

**Definition 7.9** (Derived extension). A *derived extension* is an enrichment of the AK framework in which filtered, persistence-level, or external-domain objects are interpreted inside a derived category, derived functor calculus, dg-category, triangulated category, stable homotopical category, or stable  $\infty$ -category.

**Definition 7.10** (Derived realization target). A derived extension has a *derived realization target* if it declares a functor or correspondence

$$\mathcal{R}_D : \mathcal{C}_{\text{ext}} \longrightarrow D,$$

where  $D$  is a declared derived category, dg-category, triangulated category, or stable  $\infty$ -category.

**Definition 7.11** (Core-compatible derived realization). A derived realization is *Core-compatible* if:

- (i) it has a route to a Core-readable filtered or persistence object;
- (ii) the repaired profiles

$$\text{Eval}_{i,w,\tau}(F)$$

can be evaluated or reconstructed;

- (iii) whenever the  $\text{Ext}^1$ -clause is invoked, the realized object reduces to the eligible regime of Chapter 3 and Appendix C:

$$\mathcal{R}(C_{\tau,w}F) \in D^{[-1,0]}(k\text{-mod});$$

- (iv) the edge-realization condition is declared.

**Proposition 7.12** (Derived enhancement does not replace eligibility). A *derived enhancement does not replace the eligible realization regime required for the Core  $\text{Ext}^1$ -edge*.

*Proof.* The Core  $\text{Ext}^1$ -edge theorem is proved in the specific setting

$$D^{[-1,0]}(k\text{-mod})$$

over a field, with edge realization linking  $H^{-1}$  to the repaired residual degree-1 persistence edge. A general derived enhancement need not satisfy amplitude  $[-1, 0]$ , field semisimplicity, or edge realization. Therefore eligibility remains mandatory.  $\square$

**Remark 7.13** (Derived categories beyond  $D^b(k\text{-mod})$ ). Derived categories over rings, schemes, stacks, sheaves, motives, or dg-enhanced categories may be valuable in interface appendices. However, the Chapter 3 one-way bridge does not automatically extend to them. Any such extension requires separate hypotheses and proof.

## N.4. Derived Ext-edge extensions

**Definition 7.14** (Derived Ext-edge extension). A *derived Ext-edge extension* is a proposed generalization of the Core edge identification

$$\mathrm{Ext}^1(A, k) \cong \mathrm{Hom}_k(H^{-1}(A), k)$$

to a derived category beyond  $D^b(k\text{-mod})$ .

**Definition 7.15** (Admissible derived edge extension). A derived Ext-edge extension is *admissible* only if it declares:

- (i) the ambient derived category;
- (ii) the coefficient object replacing  $k$ ;
- (iii) the amplitude condition;
- (iv) the splitting, truncation, or spectral sequence argument used to identify the edge;
- (v) the edge-realization condition linking the derived edge to repaired residual persistence data;
- (vi) all additional hypotheses needed for vanishing transport;
- (vii) artifact references or proof references supporting the extension.

**Proposition 7.16** (No automatic derived edge transport). *The Core  $\mathrm{Ext}^1$ -edge identification does not automatically transport to an arbitrary derived enhancement.*

*Proof.* The Core proof uses the semisimplicity of finite-dimensional vector spaces over a field and the amplitude condition  $[-1, 0]$ . In a general derived category, extensions may not split, higher cohomology may contribute, and the coefficient object may not be injective or projective. Thus the Core proof does not automatically apply.  $\square$

**Remark 7.17** (Spectral sequence caution). In a general derived setting, the  $\mathrm{Ext}^1$ -group may receive contributions from multiple pages or degrees of a spectral sequence. An edge map may exist, but it need not be an isomorphism without additional degeneration or vanishing hypotheses.

## N.5. Sheaf-theoretic extensions

**Definition 7.18** (Sheaf-theoretic extension). A *sheaf-theoretic extension* is an enrichment in which Core-relevant data are represented by sheaves, cosheaves, constructible sheaves, persistence sheaves, or sheaf-like objects on a topological, stratified, geometric, site-theoretic, or parameter space.

**Definition 7.19** (Sheaf realization). A *sheaf realization* is a declared functor or correspondence

$$\mathfrak{S} : \mathcal{C}_{\mathrm{ext}} \longrightarrow \mathrm{Sh}(\mathcal{X})$$

or

$$\mathfrak{S} : \mathcal{C}_{\mathrm{ext}} \longrightarrow D^b(\mathrm{Sh}(\mathcal{X})),$$

where  $\mathcal{X}$  is a declared site, topological space, stratified space, or topos.

**Definition 7.20** (Core-readable sheaf realization). A sheaf realization is *Core-readable* if it includes a declared extraction route

$$\mathfrak{E}_{\text{pers}} : \text{Sh}(\mathcal{X}) \longrightarrow \text{Pers}_k^{\text{cons}}$$

or

$$\mathfrak{E}_{\text{filt}} : \text{Sh}(\mathcal{X}) \longrightarrow \text{FiltCh}((k),$$

or an equivalent route to repaired evaluation records, so that

$$\text{Eval}_{i,W,\tau}(F)$$

can be evaluated in the Core regime.

**Proposition 7.21** (Sheaf realization does not replace persistence extraction). *A sheaf-theoretic realization does not become Core-relevant until a Core-readable persistence or filtered extraction has been declared.*

*Proof.* The Core gates are stated in terms of repaired persistence profiles, admissible representatives, eligible realized objects, tower diagnostics, and ledgers. A sheaf object by itself is not one of these. A declared extraction is needed to produce the Core-readable object or repaired evaluation record.  $\square$

**Remark 7.22** (Constructibility requirements). If sheaf-theoretic data are used to produce persistence objects, the resulting persistence objects must lie in the constructible one-parameter regime required by Appendix A. Otherwise they are not Core-readable objects.

## N.6. Sheaf gluing versus certificate-record gluing

**Definition 7.23** (Sheaf gluing claim). *A sheaf gluing claim is a statement asserting that local sheaf-theoretic objects, sections, complexes, or descent data glue to a global sheaf-theoretic object.*

**Definition 7.24** (Certificate-record gluing claim). *A certificate-record gluing claim is a statement asserting that local certificate records assemble into a globally coherent certificate record under the conditions of Chapter 5 and Appendix E.*

**Proposition 7.25** (Sheaf gluing and certificate gluing are distinct). *A certificate-record gluing theorem does not imply a sheaf gluing theorem, and a sheaf gluing theorem does not automatically imply a Core certificate-record gluing theorem.*

*Proof.* Certificate-record gluing concerns audit data: windows, thresholds, repaired evaluation records, diagnostics, budgets, gate verdicts, and artifact references. Sheaf gluing concerns descent or local-to-global reconstruction of sheaf-theoretic objects. These are different kinds of statements. One does not imply the other without additional hypotheses connecting sheaf descent to Core certificate records.  $\square$

**Remark 7.26** (Why the distinction matters). Appendix E deliberately formulated local-to-global logic at the certificate-record level. The present appendix allows sheaf-theoretic gluing as an external extension, but does not identify it with Core certificate gluing.

## N.7. Descent and stack-theoretic extensions

**Definition 7.27** (Descent datum). *A descent datum consists of local objects on a cover, isomorphisms or higher equivalences on overlaps, and coherence data on higher overlaps, sufficient to attempt reconstruction of a global object in a declared category, stack, or higher stack.*

**Definition 7.28** (Stack-theoretic extension). A *stack-theoretic extension* is an enhancement in which AK-relevant objects, filtrations, certificates, or comparison data are organized as objects of a stack, prestack, higher stack, or fibered category over a site.

**Definition 7.29** (Audit-compatible descent). A descent datum is *audit-compatible* if, in addition to satisfying the descent requirements of its ambient category, it records all Core-relevant certificate data:

- (i) declared windows;
- (ii) thresholds and collapse policies;
- (iii) repaired evaluation records;
- (iv) representative and eligibility records when invoked;
- (v) ledger entries;
- (vi) diagnostics;
- (vii) tower comparison artifacts when Type IV diagnostics are invoked;
- (viii) gate verdicts;
- (ix) overlap budgets;
- (x) artifact references.

**Proposition 7.30** (Descent is not audit by default). *A descent datum in a stack-theoretic extension is not automatically an audit-compatible certificate record.*

*Proof.* Descent data may establish object-level gluing in a stack. Audit compatibility additionally requires Core verifier-side data such as repaired evaluations, ledgers, diagnostics, gate verdicts, tower artifacts, and artifact references. These are not part of descent data by default.  $\square$

**Remark 7.31** (Descent may support, not replace, audit). A stack-theoretic descent theorem may support construction of global objects. However, Core certification still requires a certificate record satisfying the Minimal Manifest Axiom and audit discipline.

## N.8. Higher-categorical extensions

**Definition 7.32** (Higher-categorical extension). A *higher-categorical extension* is an enrichment of the AK framework using bicategories, 2-categories,  $(\infty, 1)$ -categories,  $(\infty, n)$ -categories, enriched categories, double categories, derivators, or higher toposes.

**Definition 7.33** (Higher comparison cell). A *higher comparison cell* is a 2-cell, higher cell, homotopy, equivalence, modification, or coherent family of higher morphisms comparing two or more pipeline composites.

**Definition 7.34** (Higher cell cost). A *higher cell cost* is a declared ledger-valued quantity

$$\delta(\alpha)$$

assigned to a higher comparison cell  $\alpha$ , representing the cost of using that cell in a controlled comparison.

**Definition 7.35** (Higher-cell ledger). A *higher-cell ledger* aggregates the costs of higher comparison cells:

$$\delta^{\text{higher}} := \bigoplus_{\alpha \in A^{\text{higher}}} \delta(\alpha),$$

where aggregation is performed in the declared ledger semantics.

**Proposition 7.36** (Higher cells affect audit only through ledger). *Higher comparison cells affect Core audit only through declared ledger contributions.*

*Proof.* Higher comparison cells are not Core gate verdicts and not monitored Type IV diagnostics. If they contribute discrepancy to a certified pipeline, they affect Core audit only by entering the  $\delta$ -ledger and hence the budget comparison.  $\square$

**Remark 7.37** (Relation to Appendices K and L). Appendix K treats controlled commutation and 2-cell costs in general. Appendix L specializes this to Mirror/Transfer. The present appendix allows still higher categorical versions, subject to the same ledger discipline.

## N.9. Higher functoriality and admissible representatives

**Definition 7.38** (Higher functorial representative system). A *higher functorial representative system* is a system of admissible representatives

$$C_{\tau, W}F$$

whose choices are functorial only up to coherent higher equivalences.

**Definition 7.39** (Coherence defect). A *coherence defect* is a ledger-valued cost measuring the failure of higher functoriality, associativity, unit compatibility, representative compatibility, or higher coherence to hold strictly.

**Proposition 7.40** (Higher functoriality does not strengthen representative status). *A higher functorial representative system has the same Core status as ordinary admissible representative data unless its coherence data are explicitly imported and proved as Core-compatible.*

*Proof.* Appendix B states that representatives are used through compatibility with repaired persistence profiles:

$$\mathbf{P}_i(C_{\tau, W}F) \cong \text{Eval}_{i, W, \tau}(F).$$

Higher functoriality may organize representative choices, but it does not change the required Core compatibility condition. Any nonzero coherence defect must be ledgered if used in certification.  $\square$

**Remark 7.41** (Strictness is not assumed). Higher functoriality often replaces strict equality by coherent equivalence. Such equivalence is useful, but if a certificate depends on replacing one composite by another, the relevant coherence data must be recorded and costed.

## N.10. Derived and higher Type IV interpretations

**Definition 7.42** (Higher Type IV interpretation). A *higher Type IV interpretation* is an external interpretation of monitored generic Type IV diagnostics in a derived, sheaf, stack, or higher-categorical setting.

**Definition 7.43** (Higher Type IV proxy). A *higher Type IV proxy* is a derived, sheaf-theoretic, stack-theoretic, or higher-categorical signal intended to correlate with possible monitored generic Type IV obstruction.

**Definition 7.44** (Actual Type IV diagnostic in higher context). The actual monitored generic Type IV diagnostic in a higher-categorical context is still computed from a declared tower comparison artifact

$$\phi_{i,W,\tau} : \text{ColimProfile}_{i,W,\tau}(F_\bullet) \longrightarrow \text{ApexProfile}_{i,W,\tau}(F_\infty)$$

by taking the generic fiber dimensions of its kernel and cokernel defect profiles.

**Proposition 7.45** (Higher Type IV proxy is not Type IV diagnosis). *A higher Type IV proxy does not determine whether*

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) = (0, 0)$$

or

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}) \neq (0, 0).$$

*Proof.* Core Type IV diagnostics are defined by generic fiber dimensions of kernel and cokernel defect profiles of the declared tower comparison artifact

$$\phi_{i,W,\tau}.$$

A higher-categorical signal may suggest related behavior, but it is not the diagnostic unless it explicitly computes the same defect profiles or is connected to them by a proved theorem.  $\square$

**Remark 7.46** (Higher interpretation is not replacement). A higher interpretation of Type IV may be conceptually powerful. It does not replace  $\mu$ ,  $u$ , nor does it replace the tower comparison artifact

$$\phi_{i,W,\tau}.$$

## N.11. Enhancement artifacts and manifest requirements

**Definition 7.47** (Enhancement artifact). An *enhancement artifact* is an artifact supporting a derived, sheaf, stack, or higher-categorical extension. Examples include:

- (i) derived functor definitions;
- (ii) dg-models;
- (iii) sheaf or cosheaf data;
- (iv) descent data;
- (v) stack charts;
- (vi) higher cells;
- (vii) coherence diagrams;
- (viii) comparison functors;
- (ix) extraction functors to persistence or filtered objects;
- (x) repaired evaluation records produced from the enhancement;
- (xi) tower comparison artifacts produced from the enhancement;
- (xii) enhancement-specific proof traces;



(xiii) ledger entries for higher defects.

**Definition 7.48** (Enhancement manifest entry). An *enhancement manifest entry* records:

- (i) the type of enhancement;
- (ii) the ambient category or higher structure;
- (iii) the realization target;
- (iv) the extraction route to Core-readable data;
- (v) the repaired evaluation records, if produced;
- (vi) the representative and eligibility records, if invoked;
- (vii) any tower comparison artifacts, if Type IV diagnostics are invoked;
- (viii) any higher comparison cells;
- (ix) any higher-cell ledger costs;
- (x) the status of any problem-specific implication;
- (xi) artifact references.

**Proposition 7.49** (Enhancement artifacts are required for audit readability). *A certificate record relying on a derived, sheaf, stack, or higher-categorical enhancement is not audit-readable unless it includes enhancement artifact references.*

*Proof.* Without enhancement artifacts, an auditor cannot reconstruct the ambient structure, realization target, extraction route, comparison cells, repaired evaluation records, tower artifacts, or ledger costs. Thus the certificate fails the manifest requirements of Appendix G.  $\square$

**Remark 7.50** (Enhancement manifest is additional). An enhancement manifest entry is additional to the Minimal Manifest. It does not replace declared windows, thresholds, ledgers, diagnostics, gate verdicts, or artifact references required by the Core.

## N.12. Problem-specific higher implications

**Definition 7.51** (Problem-specific higher implication). A *problem-specific higher implication* is a statement of the form:

derived/sheaf/stack/higher-categorical structure  $\implies$  problem-specific mathematical conclusion.

**Proposition 7.52** (Problem-specific higher implications are [Spec] by default). *Every problem-specific higher implication is [Spec] by default unless separately promoted and reproved under the Core guard-rails and the relevant interface protocol.*

*Proof.* By Chapter 8, problem-specific implications do not enlarge the Core by default. Derived, sheaf, stack, and higher-categorical implications typically import external semantics. Therefore they are [Spec] unless explicitly internalized and proved under the required hypotheses.  $\square$

**Remark 7.53** (No automatic domain theorem). A higher-categorical construction may be elegant and structurally meaningful. It does not by itself prove an arithmetic, PDE, geometric, categorical, motivic, sheaf-theoretic, or physical theorem. Such conclusions require explicit interface hypotheses and transport theorems.

### N.13. Importing extension results into the Core

**Definition 7.54** (Core import). A *Core import* is a process by which a result from a technical extension is promoted into the Core mathematical package.

**Definition 7.55** (Admissible Core import). A Core import is *admissible* only if:

- (i) the statement to be imported is precisely formulated;
- (ii) all hypotheses are stated in Core-compatible language;
- (iii) the proof is given under the Core guard-rails or a declared conservative extension;
- (iv) all required realization, evaluation, representative, eligibility, diagnostic, artifact, and ledger data are declared;
- (v) the import does not contradict the explicit non-claims of Chapter 7;
- (vi) the status label is updated consistently.

**Proposition 7.56** (No silent Core import). *No derived, sheaf, stack, or higher-categorical result becomes part of the Core by appearing in a technical appendix.*

*Proof.* Chapter 7 states that appendical and companion material does not enlarge the Core unless explicitly imported and reproved. The present appendix is technical extension material. Therefore its results remain non-Core unless an admissible Core import is performed.  $\square$

**Remark 7.57** (Import is exceptional). Core import should be exceptional. The default status of material in this appendix is Technical Extension or [Spec], not Core theorem.

### N.14. Compatibility with ledger semantics

**Definition 7.58** (Higher-extension ledger component). A *higher-extension ledger component* is a declared budget contribution arising from derived, sheaf, descent, stack, or higher-categorical comparison data. It may include:

$$\delta^{\text{der}}, \quad \delta^{\text{sh}}, \quad \delta^{\text{desc}}, \quad \delta^{\text{stack}}, \quad \delta^{\text{higher}}.$$

**Definition 7.59** (Higher-extension adjusted budget). If a certificate relies on higher-extension data, its adjusted budget is

$$\Sigma\delta_{\text{higher}} := \Sigma\delta_{\text{base}} \oplus \delta^{\text{der}} \oplus \delta^{\text{sh}} \oplus \delta^{\text{desc}} \oplus \delta^{\text{stack}} \oplus \delta^{\text{higher}},$$

with all omitted components set to the ledger identity.

**Proposition 7.60** (Higher-extension evidence must pass adjusted budget). *A certificate record relying on nonzero higher-extension defects may pass Audit Mode only if*

$$\Sigma\delta_{\text{higher}} < \text{gap}_{\tau}.$$

*Proof.* All defects used by a certificate must enter the  $\delta$ -ledger. If higher-extension defects are used, they contribute to the adjusted total budget. Audit Mode requires the strict budget comparison.  $\square$

**Remark 7.61** (No budget-free higher coherence). Higher coherence data may be elegant, but nonzero coherence defects are not budget-free. If they are used in certification, they must be ledgered.

## N.15. Summary of higher extension discipline

**Theorem 7.62** (Appendix N higher-extension package). *Within the Technical Extension layer, the following package is available.*

- (i) *Derived, sheaf, stack, and higher-categorical enhancements may organize external structures around the Core.*
- (ii) *Such enhancements do not update Core verdicts unless translated into verifier-side Core data.*
- (iii) *Derived enhancements do not replace the eligible realization regime.*
- (iv) *General derived Ext-edge extensions require separate hypotheses and proof.*
- (v) *Sheaf realizations become Core-relevant only through declared persistence or filtered extraction.*
- (vi) *Sheaf gluing and certificate-record gluing are distinct.*
- (vii) *Descent data are not audit-compatible by default.*
- (viii) *Higher comparison cells affect Core audit only through declared ledger contributions.*
- (ix) *Higher functoriality does not strengthen representative status unless imported and proved.*
- (x) *Higher Type IV proxies do not replace monitored Type IV diagnostics or the tower comparison artifact*

$$\phi_{i,W,\tau}.$$

- (xi) *Enhancement artifacts are required when enhancements support certificate records.*
- (xii) *Problem-specific higher implications are [Spec] by default.*
- (xiii) *No technical extension result enters the Core without an admissible Core import.*
- (xiv) *Higher-extension defects must satisfy the adjusted budget condition*

$$\Sigma\delta_{\text{higher}} < \text{gap}_{\tau}.$$

*Proof.* Items (i)–(ii) follow from the external enhancement principle. Items (iii)–(iv) follow from the derived extension and derived edge propositions. Items (v)–(vi) follow from the sheaf realization and gluing distinction. Item (vii) follows from the descent-not-audit proposition. Items (viii)–(ix) follow from the higher-cell and higher-functoriality propositions. Item (x) is Proposition 7.45. Item (xi) is Proposition 7.49. Item (xii) is Proposition 7.52. Item (xiii) is Proposition 7.56. Item (xiv) is Proposition 7.60.  $\square$

## N.16. Explicit exclusions

The following are not asserted in Appendix N.

- No derived enhancement is treated as part of the Core by default.
- No sheaf-theoretic enhancement is treated as part of the Core by default.
- No stack-theoretic descent datum is treated as a Core certificate by default.
- No higher-categorical coherence datum is treated as strict equality unless explicitly declared and justified.

- No derived Ext-edge theorem beyond  $D^{[-1,0]}(k\text{-mod})$  is asserted without additional proof.
- No sheaf gluing theorem is inferred from certificate-record gluing.
- No certificate-record gluing theorem is inferred from sheaf gluing.
- No higher Type IV proxy replaces

$$(\mu_{\text{Collapse}}, u_{\text{Collapse}}).$$

- No higher Type IV proxy replaces the tower comparison artifact

$$\phi_{i,W,\tau}.$$

- No higher comparison cell replaces B-Gate<sup>+</sup>.
- No higher comparison cell replaces the repaired topological condition

$$\text{Eval}_{1,W,\tau}(F) = 0.$$

- No higher comparison cell replaces the budget condition

$$\Sigma\delta < \text{gap}_{\tau}.$$

- No higher functorial representative system has stronger Core status than an admissible representative system unless imported and proved.
- No admissible representative

$$C_{\tau,W}F$$

is replaced by higher coherence data.

- No problem-specific arithmetic, PDE, geometric, categorical, motivic, sheaf-theoretic, stack-theoretic, or higher-categorical conclusion is proved here.
- No technical extension appendix enlarges the Core without an admissible Core import.

## N.17. Closure of Part III

This appendix closes the Technical Extension Appendices. The role of Part III is to provide optional technical layers:

advisory indicators,    discretization stability,    measurement stability,  
 controlled commutation,    Mirror/Transfer ledgers,  
 ledger catalogs,    derived/sheaf/stack/higher enhancements.

None of these layers changes the Core theorem package.

**Theorem 7.63** (Part III closure principle). *The Technical Extension Appendices H–N provide optional technical infrastructure around the AK Core. They do not enlarge the Core mathematical package, do not replace Core gates, do not replace repaired evaluation records, do not replace monitored Type IV diagnostics, do not replace required tower comparison artifacts, and do not alter the audit budget condition.*

*Proof.* Appendices H–N each state their non-Core technical-extension status and their explicit exclusions. Each extension is subordinated to Core sovereignty, manifest requirements, repaired evaluation discipline, representative discipline, diagnostic discipline, tower artifact discipline, and ledger-based audit. Therefore Part III as a whole is technical infrastructure, not a Core enlargement.  $\square$

**Remark 7.64** (Transition to problem interfaces). After Part III, the natural next layer is the Problem Interface Appendices. Those appendices may connect the Core and technical extensions to arithmetic, PDE, geometric, categorical, or other domain-specific programs. By Chapter 8, such connections are [Spec] by default unless explicitly promoted and reproved.

## N.18. Provenance and status

The material in this appendix depends on:

- (i) the Core non-claim discipline of Chapter 7;
- (ii) the interface protocol of Chapter 8;
- (iii) the repaired evaluation discipline of Appendix A;
- (iv) the admissible representative discipline of Appendix B;
- (v) the eligible realization regime of Appendix C;
- (vi) the monitored Type IV diagnostics and tower comparison artifact discipline of Appendix D;
- (vii) the certificate-record gluing discipline of Appendix E;
- (viii) the manifest requirements of Appendix G;
- (ix) the advisory and ledger disciplines of Appendices H–M.

Its role is to close the technical extension layer by allowing higher structures while preventing silent Core enlargement.

**Status labels for Appendix N.** *Technical extension definitions:* Definitions [7.4](#), [7.5](#), [7.6](#), [7.9](#), [7.10](#), [7.11](#), [7.14](#), [7.15](#), [7.18](#), [7.19](#), [7.20](#), [7.23](#), [7.24](#), [7.27](#), [7.28](#), [7.29](#), [7.32](#), [7.33](#), [7.34](#), [7.35](#), [7.38](#), [7.39](#), [7.42](#), [7.43](#), [7.44](#), [7.47](#), [7.48](#), [7.51](#), [7.54](#), [7.55](#), [7.58](#), [7.59](#).

*Technical extension propositions/theorems:* Propositions [7.7](#), [7.12](#), [7.16](#), [7.21](#), [7.25](#), [7.30](#), [7.36](#), [7.40](#), [7.45](#), [7.49](#), [7.52](#), [7.56](#), [7.60](#), Theorems [7.62](#), [7.63](#).

*Operational cautions:* Remarks [7.1](#), [7.2](#), [7.3](#), [7.8](#), [7.13](#), [7.17](#), [7.22](#), [7.26](#), [7.31](#), [7.37](#), [7.41](#), [7.46](#), [7.50](#), [7.53](#), [7.57](#), [7.61](#), [7.64](#).

*Explicit exclusions:* Section N.16.

*Part closure:* Theorem [7.63](#).